

Notebook ICPC

La Tasita

Team: UPC - Turistas

Integrantes

spigi
Jean_Pool
Santi2007939
Chatito17

Índice

1. Dynamic Programming	3
1.1. Convex Hull Trick	3
1.2. Data Structure Optimization	3
1.3. Digit Dp	4
1.4. Divide And Conquer	5
1.5. Knapsack	6
1.6. Knapsack Binary	6
1.7. Knapsack Re	6
1.8. Knapsack Subset Sum	7
1.9. Knuth	7
1.10. LCS	7
1.11. LIS	8
1.12. Lisfast	8
1.13. Maximum Subarray	9
1.14. Subset Sum Bitset	9
2. Data Structures	10
2.1. Binary Lifting Weighted	10
2.2. Binary Lifting	11
2.3. DSU Rollback	11
2.4. DSU	11
2.5. Fenwick Tree 2D Range	12
2.6. Fenwick Tree 2D	12
2.7. Fenwick Tree Range	12
2.8. Fenwick Tree	13
2.9. HLD	13
2.10. Ordered Set	14
2.11. Persistent Segment Tree	15
2.12. Segment Tree Lazy	16
2.13. Segment Tree	16
2.14. Sparse Table	17
2.15. Two Pointers	17
3. Geometry	18
3.1. Convex Hull	18
4. Graphs	19
4.1. 2-SAT	19
4.2. Bellman Ford	21
4.3. Cd	21
4.4. Centroid Decomposition	23
4.5. Dijkstra	24
4.6. Floyd Warshall	25
4.7. Ford Fulkerson	25
4.8. Functional Graph	27
4.9. Hungarian	28
4.10. Kosaraju Scc	30
4.11. Kruskal	32
4.12. Topological Sort	33
5. Math	33
5.1. Binary Exponentiation	33
5.2. Expected Value	34
5.3. FFT	35
5.4. Matrix Exponentiation	36
5.5. NTT	37

6. Strings	38
6.1. Aho Corasick	38
6.2. Bit Trie	39
6.3. Double Hash	39
6.4. Hash	40
6.5. KMP Automaton	41
6.6. KMP	42
6.7. Rolling Hash	43
6.8. String Hashing	43
6.9. Suffix Array	44
6.10. Suffix Tree	45
6.11. Trie	48
6.12. Z Function	48

Dynamic Programming

Convex Hull Trick

```
1 // Convex Hull Trick (CHT) - O(1) amortizado por insercion y query
2 // Optimiza: dp[i] = min/max ( m_j * x_i + c_j )
3 // Condicion: Pendientes 'm' insertadas en orden MONOTONO (creciente/decreciente).
4 // Para queries 'x' monotonos: query() usa dos punteros O(1).
5 // Para queries 'x' arbitrarios: query_bin() usa busqueda binaria O(log N).
6
7 struct Line {
8     ll m, c;
9     ll eval(ll x) const { return m * x + c; }
10 };
11
12 // CHT para MINIMIZAR (m decreciente, queries x crecientes)
13 // Para MAXIMIZAR: invertir signos o usar m creciente y cambiar condicion is_bad
14 struct CHT {
15     vector<Line> hull;
16     int ptr = 0;
17
18     // Retorna true si l2 es redundante entre l1 y l3
19     bool is_bad(const Line& l1, const Line& l2, const Line& l3) {
20         // Interseccion(l1, l2) >= Interseccion(l2, l3) usando __int128 para evitar overflow
21         return (__int128)(l1.c - l2.c) * (l3.m - l2.m) >= (__int128)(l2.c - l3.c) * (l2.m - l1.m);
22     }
23
24     void add(ll m, ll c) {
25         Line l = {m, c};
26         while (sz(hull) >= 2 && is_bad(hull[sz(hull) - 2], hull.back(), l)) {
27             hull.pop_back();
28         }
29         hull.pb(l);
30         if (ptr >= sz(hull)) ptr = sz(hull) - 1;
31     }
32
33     // Query O(1) amortizado (requiere x monotonamente creciente)
34     ll query(ll x) {
35         if (hull.empty()) return LINF;
36         while (ptr + 1 < sz(hull) && hull[ptr + 1].eval(x) <= hull[ptr].eval(x)) {
37             ++ptr;
38         }
39         return hull[ptr].eval(x);
40     }
41
42     // Query O(log N) para x arbitrarios (no requiere que x sea monotono)
43     ll query_bin(ll x) {
44         if (hull.empty()) return LINF;
45         int l = 0, r = sz(hull) - 1;
46         while (l < r) {
47             int mid = l + (r - l) / 2;
48             if (hull[mid + 1].eval(x) <= hull[mid].eval(x)) l = mid + 1;
49             else r = mid;
50         }
51         return hull[l].eval(x);
52     }
53 };
```

Data Structure Optimization

```
1 // DP Optimization with Data Structures (Segment Tree / Fenwick) - O(N log N)
2 // Optimiza transiciones: dp[i] = max_{j < i, L <= val[j] <= R} { dp[j] } + cost[i]
3
4 // --- 1. Compresion de Coordenadas -----
5 // Mapea valores arbitrarios a [0, sz(vals)-1]
6 template<class T>
7 vi compress(const vector<T>& a) {
8     vector<T> vals = a;
9     sort(all(vals));
10    vals.erase(unique(all(vals)), vals.end());
11    vi res(sz(a));
12    rep(i, sz(a)) {
13        res[i] = lower_bound(all(vals), a[i]) - vals.begin();
14    }
15    return res;
16 }
17
18 // --- 2. Segment Tree para Maximos en Rango -----
19 struct SegTreeDP {
20     int n;
21     vll tree;
```

```

22 SegTreeDP(int n) : n(n), tree(4 * n, 0) {}
23
24 void update(int pos, ll val, int node, int l, int r) {
25     if (l == r) {
26         ckmax(tree[node], val);
27         return;
28     }
29     int mid = l + (r - l) / 2;
30     if (pos <= mid) update(pos, val, 2 * node, l, mid);
31     else update(pos, val, 2 * node + 1, mid + 1, r);
32     tree[node] = max(tree[2 * node], tree[2 * node + 1]);
33 }
34 void update(int pos, ll val) { update(pos, val, 1, 0, n - 1); }
35
36 ll query(int ql, int qr, int node, int l, int r) {
37     if (ql > r || qr < l) return 0; // valor neutro
38     if (ql <= l && r <= qr) return tree[node];
39     int mid = l + (r - l) / 2;
40     return max(query(ql, qr, 2 * node, l, mid),
41               query(ql, qr, 2 * node + 1, mid + 1, r));
42 }
43 ll query(int ql, int qr) {
44     if (ql > qr) return 0;
45     return query(ql, qr, 1, 0, n - 1);
46 }
47 };
48
49 // --- 3. Fenwick de Maximos (O(log N) - Solo para Prefijos [0..idx]) -----
50 struct BITMax {
51     int n;
52     vll bit;
53     BITMax(int n) : n(n), bit(n + 1, 0) {}
54     void update(int i, ll val) {
55         for (++i; i <= n; i += i & -i) ckmax(bit[i], val);
56     }
57     ll query(int i) { // maximo en [0..i]
58         ll res = 0;
59         for (++i; i > 0; i -= i & -i) ckmax(res, bit[i]);
60         return res;
61     }
62 };
63
64 // Ejemplo de uso (LIS Ponderado / Flowers):
65 // vi comp = compress(alturas);
66 // SegTreeDP st(sz(comp));
67 // rep(i, n) {
68 //     ll best_prev = st.query(0, comp[i] - 1);
69 //     st.update(comp[i], best_prev + bellezas[i]);
70 // }
71 // ll ans = st.query(0, sz(comp) - 1);

```

Digit Dp

```

1 // Digit DP Template - Complejidad O(Longitud * Estados * 10)
2 // Estados tipicos: (pos, tight, leading_zero, estado_personalizado)
3
4 // --- 1. Enfoque Estandar f(R) - f(L - 1) -----
5 string S;
6 ll memo[20][2][2][100]; // Ajustar dimensiones del estado personalizado
7
8 ll digitDP(int pos, bool tight, bool lz, int state) {
9     if (pos == sz(S)) {
10         // Retornar 1 si el 'state' es valido, 0 si no
11         return 1;
12     }
13     ll& ans = memo[pos][tight][lz][state];
14     if (ans != -1) return ans;
15
16     ans = 0;
17     int limit = tight ? (S[pos] - '0') : 9;
18
19     for (int dig = 0; dig <= limit; ++dig) {
20         bool next_tight = tight && (dig == limit);
21         bool next_lz = lz && (dig == 0);
22
23         // Actualizar estado personalizado (ejemplo: suma de digitos, bitmask, mod, etc.)
24         int next_state = next_lz ? 0 : (state + dig);
25
26         ans += digitDP(pos + 1, next_tight, next_lz, next_state);
27         // Si pide modulo: ans %= MOD;
28     }
29     return ans;
30 }

```

```

31
32 ll countValid(ll n) {
33     if (n < 0) return 0;
34     S = to_string(n);
35     memset(memo, -1, sizeof(memo));
36     return digitDP(0, true, true, 0);
37 }
38
39 // Rango [L, R]
40 ll solveRange(ll L, ll R) {
41     return countValid(R) - countValid(L - 1);
42 }
43
44 // --- 2. Enfoque Simultaneo [L, R] (Tight Inferior y Superior) -----
45 // string A, B; // Rellenar A con '0' a la izquierda hasta que sz(A) == sz(B)
46 // ll dp2[20][2][2][2]; // pos, tight_low, tight_high, leading_zero
47 // int min_d = tight_low ? (A[pos] - '0') : 0;
48 // int max_d = tight_high ? (B[pos] - '0') : 9;
49 // for (int d = min_d; d <= max_d; ++d) {
50 //     bool n_low = tight_low && (d == min_d);
51 //     bool n_high = tight_high && (d == max_d);
52 //     bool n_lz = lz && (d == 0);
53 //     ...
54 // }

```

Divide And Conquer

```

1 // Divide and Conquer DP Optimization - O(K * N log N)
2 // Condicion: opt[k][i] <= opt[k][i+1] (e.g., Quadrangle Inequality)
3 // Recurrencia: dp[k][i] = min_{j < i} { dp[k-1][j] + cost(j, i) }
4
5 vll dp_prev, dp_curr;
6
7 // 1. Costo O(1) con formulas / prefijos (personalizar segun problema)
8 ll cost(int j, int i) {
9     // Ejemplo: (pref[i] - pref[j])^2
10    // ll s = pref[i] - pref[j];
11    // return s * s;
12    return 0;
13 }
14
15 // 2. Variante: Costo con Ventana Dinamica / Two Pointers estilo Mo
16 // int curL = 1, curR = 0; ll cur_cost = 0;
17 // void move_to(int L, int R) {
18 //     while (curR < R) add(++curR);
19 //     while (curL > L) add(--curL);
20 //     while (curR > R) remove(curR--);
21 //     while (curL < L) remove(curL++);
22 // }
23
24 void compute(int l, int r, int optL, int optR) {
25     if (l > r) return;
26     int mid = l + (r - l) / 2;
27     int best_opt = -1;
28     dp_curr[mid] = LINF;
29     int limit = min(mid - 1, optR);
30
31     for (int j = optL; j <= limit; ++j) {
32         if (dp_prev[j] == LINF) continue;
33         // Si se usa ventana dinamica: move_to(j + 1, mid);
34         ll val = dp_prev[j] + cost(j, mid); // o dp_prev[j] + cur_cost;
35         if (val < dp_curr[mid]) {
36             dp_curr[mid] = val;
37             best_opt = j;
38         }
39     }
40
41     compute(l, mid - 1, optL, best_opt);
42     compute(mid + 1, r, best_opt, optR);
43 }
44
45 // Resuelve para K particiones en un arreglo de tamaño N (1-indexed)
46 ll solveDCDP(int n, int k) {
47     dp_prev.assign(n + 1, LINF);
48     dp_curr.assign(n + 1, LINF);
49
50     // Caso base k = 1
51     dp_prev[0] = 0;
52     repl(i, n) dp_prev[i] = cost(0, i);
53
54     // Transicion para k = 2..K
55     for (int c = 2; c <= k; ++c) {
56         compute(1, n, 0, n - 1);

```

```

57     dp_prev = dp_curr;
58 }
59 return dp_prev[n];
60 }

```

Knapsack

```

1 // 0/1 Knapsack - O(N * W)
2 // items: {peso, valor}
3 ll knapsack(const vii& items, int W) {
4     vll dp(W + 1, -1);
5     dp[0] = 0;
6     each(item, items) {
7         int w = item.ff;
8         ll v = item.ss;
9         for (int j = W; j >= w; --j) {
10             if (dp[j - w] != -1) {
11                 ckmax(dp[j], dp[j - w] + v);
12             }
13         }
14     }
15     ll max_v = 0;
16     rep(j, W + 1) ckmax(max_v, dp[j]);
17     return max_v;
18 }

```

Knapsack Binary

```

1 // Bounded Knapsack (Binary Splitting) - O(W * sum(log K))
2 ll knapsackBounded(const vi& pesos, const vll& valores, const vi& copias, int W) {
3     vii items;
4     int n = sz(pesos);
5     rep(i, n) {
6         int k = copias[i], b = 1;
7         while (k > 0) {
8             int num = min(k, b);
9             items.eb(pesos[i] * num, valores[i] * num);
10            k -= num;
11            b *= 2;
12        }
13    }
14    vll dp(W + 1, -1);
15    dp[0] = 0;
16    each(item, items) {
17        int w = item.ff;
18        ll v = item.ss;
19        for (int j = W; j >= w; --j) {
20            if (dp[j - w] != -1) {
21                ckmax(dp[j], dp[j - w] + v);
22            }
23        }
24    }
25    ll ans = 0;
26    rep(j, W + 1) ckmax(ans, dp[j]);
27    return ans;
28 }

```

Knapsack Re

```

1 // 0/1 Knapsack with Reconstruction - O(N * W)
2 // items: {peso, valor} -> retorna {max_valor, indices_tomados}
3 pair<ll, vi> knapsackReconstruction(const vii& items, int W) {
4     int n = sz(items);
5     vvll dp(n + 1, vll(W + 1, 0));
6     rep(i, n) {
7         int w = items[i].ff;
8         ll v = items[i].ss;
9         rep(j, W + 1) {
10             dp[i + 1][j] = dp[i][j];
11             if (j >= w) ckmax(dp[i + 1][j], dp[i][j - w] + v);
12         }
13     }
14     vi taken;
15     int cur_w = W;
16     for (int i = n - 1; i >= 0; --i) {
17         int w = items[i].ff;
18         ll v = items[i].ss;
19         if (cur_w >= w && dp[i + 1][cur_w] == dp[i][cur_w - w] + v) {
20             taken.pb(i);
21             cur_w -= w;
22         }
23     }
24 }

```

```

24     reverse(all(taken));
25     return {dp[n][W], taken};
26 }

```

Knapsack Subset Sum

```

1 // Subset Sum with Equal Partition / Difference DP - O(N * sum)
2 // Encuentra la máxima altura igual de dos subconjuntos disjuntos (Tallest Billboard)
3 int knapsackDiferencia(const vi& arr) {
4     int sum = 0;
5     each(x, arr) sum += x;
6     vi dp(sum + 1, -1);
7     dp[0] = 0;
8
9     each(h, arr) {
10        vi next_dp = dp;
11        rep(d, sum + 1) {
12            if (dp[d] < 0) continue;
13            // 1. No usar h (ya cubierto por next_dp = dp)
14            // 2. Añadir al lado mayor
15            if (d + h <= sum) ckmax(next_dp[d + h], dp[d]);
16            // 3. Añadir al lado menor
17            int diff = abs(d - h);
18            ckmax(next_dp[diff], dp[d] + min(d, h));
19        }
20        dp = move(next_dp);
21    }
22    return dp[0];
23 }

```

Knuth

```

1 // Knuth's DP Optimization - O(N^2)
2 // Recurrencia: dp[i][j] = min_{i <= k < j} { dp[i][k] + dp[k+1][j] } + cost(i, j)
3 // Condicion de optimo: opt[i][j-1] <= opt[i][j] <= opt[i+1][j]
4 // Requiere que cost(i, j) cumpla Quadrangle Inequality y Monotonicidad
5
6 ll knuthDP(const vll& arr) {
7     int n = sz(arr);
8     // Prefijos para calcular costo de rango en O(1)
9     vll pref(n + 1, 0);
10    rep(i, n) pref[i + 1] = pref[i] + arr[i];
11
12    auto cost = [&](int i, int j) -> ll {
13        return pref[j + 1] - pref[i];
14    };
15
16    vvll dp(n, vll(n, 0));
17    vvi opt(n, vi(n, 0));
18
19    // Casos base longitud 1
20    rep(i, n) {
21        dp[i][i] = 0;
22        opt[i][i] = i;
23    }
24
25    // Iterar por longitud del intervalo len = 2..N
26    // 0 equivalentemente i de n-2 hasta 0, j de i+1 hasta n-1
27    for (int len = 2; len <= n; ++len) {
28        for (int i = 0; i <= n - len; ++i) {
29            int j = i + len - 1;
30            dp[i][j] = LINF;
31            ll c = cost(i, j);
32
33            int l_opt = opt[i][j - 1];
34            int r_opt = (i + 1 <= j) ? opt[i + 1][j] : j - 1;
35            r_opt = min(r_opt, j - 1);
36
37            for (int k = l_opt; k <= r_opt; ++k) {
38                ll val = dp[i][k] + dp[k + 1][j] + c;
39                if (val < dp[i][j]) {
40                    dp[i][j] = val;
41                    opt[i][j] = k;
42                }
43            }
44        }
45    }
46    return dp[0][n - 1];
47 }

```

LCS

```

1 // Longest Common Subsequence & Reconstruction - O(N * M)
2 pair<int, string> LCS(const string& s1, const string& s2) {
3     int n = sz(s1), m = sz(s2);
4     vvi dp(n + 1, vi(m + 1, 0));
5     rep(i, n) rep(j, m) {
6         if (s1[i] == s2[j]) dp[i + 1][j + 1] = dp[i][j] + 1;
7         else dp[i + 1][j + 1] = max(dp[i][j + 1], dp[i + 1][j]);
8     }
9     string ans = "";
10    int i = n, j = m;
11    while (i > 0 && j > 0) {
12        if (s1[i - 1] == s2[j - 1]) {
13            ans += s1[i - 1];
14            i--; j--;
15        } else if (dp[i - 1][j] >= dp[i][j - 1]) {
16            i--;
17        } else {
18            j--;
19        }
20    }
21    reverse(all(ans));
22    return {dp[n][m], ans};
23 }

```

LIS

```

1 // O(N^2) Longest Increasing Subsequence
2 int LIS(const vi& arr) {
3     int n = sz(arr);
4     vi dp(n, 1);
5     int res = 0;
6     rep(i, n) {
7         rep(j, i) {
8             if (arr[j] < arr[i]) ckmax(dp[i], dp[j] + 1);
9         }
10        ckmax(res, dp[i]);
11    }
12    return res;
13 }
14
15 vi LISReconstruccion(const vi& arr) {
16     int n = sz(arr);
17     vi dp(n, 1), padre(n, -1);
18     int res = 0, best_idx = -1;
19
20     rep(i, n) {
21         rep(j, i) {
22             if (arr[j] < arr[i] && dp[j] + 1 > dp[i]) {
23                 dp[i] = dp[j] + 1;
24                 padre[i] = j;
25             }
26         }
27         if (dp[i] > res) {
28             res = dp[i];
29             best_idx = i;
30         }
31     }
32
33     vi secuencia;
34     for (int i = best_idx; i != -1; i = padre[i]) {
35         secuencia.pb(arr[i]);
36     }
37     reverse(all(secuencia));
38     return secuencia;
39 }

```

Lisfast

```

1 // LIS Fast O(N log N)
2 int LISFast(const vi& arr) {
3     vi res;
4     each(x, arr) {
5         auto it = lower_bound(all(res), x); // upper_bound para no estrictamente creciente
6         if (it == res.end()) res.pb(x);
7         else *it = x;
8     }
9     return sz(res);
10 }
11
12 // LIS Fast con Reconstrucción - O(N log N)
13 vi LISFastReconstruccion(const vi& arr) {
14     int n = sz(arr);

```

```

15     vi res, resIndex, padre(n, -1);
16
17     rep(i, n) {
18         int x = arr[i];
19         auto it = lower_bound(all(res), x);
20         int pos = it - res.begin();
21
22         if (it == res.end()) {
23             res.pb(x);
24             resIndex.pb(i);
25         } else {
26             *it = x;
27             resIndex[pos] = i;
28         }
29
30         if (pos > 0) padre[i] = resIndex[pos - 1];
31     }
32
33     vi lis;
34     int idx = resIndex.back();
35     while (idx != -1) {
36         lis.pb(arr[idx]);
37         idx = padre[idx];
38     }
39     reverse(all(lis));
40     return lis;
41 }

```

Maximum Subarray

```

1 // Kadane's Algorithm - Maximum Subarray Sum O(N)
2 ll maximumSub(const vll& arr) {
3     ll cur = 0, ans = -LINF;
4     each(x, arr) {
5         cur = max(x, cur + x);
6         ckmax(ans, cur);
7     }
8     return ans;
9 }

```

Subset Sum Bitset

```

1 // Subset Sum con Bitset & Binary Splitting - O((S / 64) * sum(log count))
2 // 1. Bitset acelera transiciones booleanas dividiendo entre 64.
3 // 2. Binary splitting agrupa elementos repetidos: si 'val' aparece C veces,
4 //    se descompone en potencias de 2: 1*val, 2*val, 4*val, ..., R*val.
5
6 const int MAX_SUM = 500005;
7
8 // --- 1. Bounded Subset Sum con Bitset Estandar -----
9 bitset<MAX_SUM> subsetSum(const vi& arr, int target) {
10     // 1. Contar frecuencias
11     map<int, int> freq;
12     each(x, arr) if (x <= target) freq[x]++;
13
14     // 2. Descomposicion binaria de frecuencias
15     vi items;
16     for (auto& [val, count] : freq) {
17         int c = min(count, target / val);
18         for (int k = 1; k <= c; k *= 2) {
19             items.pb(val * k);
20             c -= k;
21         }
22         if (c > 0) items.pb(val * c);
23     }
24
25     // 3. DP con Bitset
26     bitset<MAX_SUM> dp;
27     dp[0] = 1;
28     each(w, items) {
29         dp |= (dp << w);
30     }
31     return dp;
32 }
33
34 // --- 2. Bitset con Tamaño Dinamico (Template Recursivo) -----
35 // Ejecuta con bitset<len> ajustado al target para no desperdiciar operaciones
36 int reachable_max = 0;
37 template <int len = 1>
38 void dynamicSubsetSum(int target, const vi& items) {
39     if (target >= len) {
40         dynamicSubsetSum<min(len * 2, MAX_SUM)>(target, items);

```

```

41     return;
42 }
43 bitset<len> dp;
44 dp[0] = 1;
45 each(x, items) dp |= (dp << x);
46
47 for (int i = target; i >= 0; --i) {
48     if (dp[i]) {
49         reachable_max = i;
50         break;
51     }
52 }
53 }
54
55 // Ejemplo de uso: maxima suma posible <= target con binary splitting
56 int getMaxSubsetSum(const vi& valores, int target) {
57     map<int, int> freq;
58     each(x, valores) if (x <= target) freq[x]++;
59     vi items;
60     for (auto& [val, count] : freq) {
61         int c = min(count, target / val);
62         for (int k = 1; k <= c; k *= 2) {
63             items.pb(val * k);
64             c -= k;
65         }
66         if (c > 0) items.pb(val * c);
67     }
68     reachable_max = 0;
69     dynamicSubsetSum(target, items);
70     return reachable_max;
71 }
72
73
74 // --- 3. Reconstruccion de Elementos Tomados -----
75 vi reconstructSubsetSum(const vi& arr, int target) {
76     int n = sz(arr);
77     // Para reconstruccion exacta se pueden guardar bitsets por capas o usar backtrack
78     vector<bitset<MAX_SUM>> dp(n + 1);
79     dp[0][0] = 1;
80     rep(i, n) {
81         dp[i + 1] = dp[i] | (dp[i] << arr[i]);
82     }
83     if (!dp[n][target]) return {}; // Imposible
84
85     vi taken;
86     int cur = target;
87     for (int i = n - 1; i >= 0; --i) {
88         if (cur >= arr[i] && dp[i][cur - arr[i]]) {
89             taken.pb(arr[i]);
90             cur -= arr[i];
91         }
92     }
93     return taken;
94 }

```

Data Structures

Binary Lifting Weighted

```

1 struct BinLiftW {
2     int n, K;
3     vector<vector<pair<int, ll>>> g;
4     vector<vector<int>> up;
5     vector<vector<ll>> mx; // mx[k][v] = max edge weight on the 2^k jump from v
6     vector<int> dep;
7     ll merge(ll a, ll b) { return max(a, b); } // your merge here
8     const ll ID = LLONG_MIN; // identity of merge
9     BinLiftW(int n) : n(n), K(_lg(n)+1), g(n), up(K, vector<int>(n)), mx(K, vector<ll>(n, LLONG_MIN)),
10         dep(n) {}
11     void add_edge(int a, int b, ll w) { g[a].push_back({b, w}); g[b].push_back({a, w}); }
12     void build(int root = 0) {
13         vector<int> s = {root};
14         up[0][root] = root;
15         while (!s.empty()) {
16             int u = s.back(); s.pop_back();
17             for (auto [v, w] : g[u]) if (v != up[0][u]) { up[0][v] = u; mx[0][v] = w; dep[v] = dep[u]+1; s.push_back(v); }
18         }
19         for (int k = 1; k < K; k++)
20             for (int v = 0; v < n; v++) {
21                 int m = up[k-1][v];

```

```

21         up[k][v] = up[k-1][m];
22         mx[k][v] = merge(mx[k-1][v],mx[k-1][m]);
23     }
24 }
25 ll query(int a, int b) { // aggregate over edges of the path a-b
26     ll res = ID;
27     if (dep[a] < dep[b]) swap(a,b);
28     for (int k = 0; k < K; k++)
29         if ((dep[a]-dep[b])>>k & 1) { res = merge(res,mx[k][a]); a = up[k][a]; }
30     if (a == b) return res;
31     for (int k = K-1; k >= 0; k--)
32         if (up[k][a] != up[k][b]) {
33             res = merge(res,merge(mx[k][a],mx[k][b]));
34             a = up[k][a]; b = up[k][b];
35         }
36     return merge(res,merge(mx[0][a],mx[0][b]));
37 }
38 };

```

Binary Lifting

```

1 struct BinLift {
2     int n, K;
3     vector<vector<int>> g, up;
4     vector<int> dep;
5     BinLift(int n) : n(n), K(_lg(n)+1), g(n), up(K,vector<int>(n)), dep(n) {}
6     void add_edge(int a, int b) { g[a].push_back(b); g[b].push_back(a); }
7     void build(int root = 0) {
8         vector<int> s = {root};
9         up[0][root] = root; // root is its own parent
10        while (!s.empty()) { // iterative dfs
11            int u = s.back(); s.pop_back();
12            for (int v : g[u]) if (v != up[0][u]) { up[0][v] = u; dep[v] = dep[u]+1; s.push_back(v); }
13        }
14        for (int k = 1; k < K; k++)
15            for (int v = 0; v < n; v++) up[k][v] = up[k-1][up[k-1][v]];
16    }
17    int kth(int u, int k) { // k-th ancestor of u, root if k > dep[u]
18        for (int i = 0; i < K; i++) if (k>>i & 1) u = up[i][u];
19        return u;
20    }
21    int lca(int a, int b) {
22        if (dep[a] < dep[b]) swap(a,b);
23        a = kth(a,dep[a]-dep[b]);
24        if (a == b) return a;
25        for (int k = K-1; k >= 0; k--)
26            if (up[k][a] != up[k][b]) { a = up[k][a]; b = up[k][b]; }
27        return up[0][a];
28    }
29    int dist(int a, int b) { return dep[a]+dep[b]-2*dep[lca(a,b)]; }
30 };

```

DSU Rollback

```

1 struct DSURollback {
2     int n, comps;
3     vector<int> p, siz;
4     vector<pair<int,int>> st; // history: (child root, parent root), (-1,-1) if no merge
5     DSURollback(int n) : n(n), comps(n), p(n), siz(n,1) { iota(p.begin(),p.end(),0); }
6     int find(int x) { while (p[x] != x) x = p[x]; return x; } // no path compression
7     bool unite(int a, int b) {
8         a = find(a); b = find(b);
9         if (a == b) { st.push_back({-1,-1}); return false; }
10        if (siz[a] < siz[b]) swap(a,b);
11        p[b] = a; siz[a] += siz[b]; comps--;
12        st.push_back({b,a});
13        return true;
14    }
15    void rollback() { // undoes the last unite (even if it did not merge)
16        auto [b,a] = st.back(); st.pop_back();
17        if (b < 0) return;
18        p[b] = b; siz[a] -= siz[b]; comps++;
19    }
20    bool same(int a, int b) { return find(a) == find(b); }
21    int size(int x) { return siz[find(x)]; }
22 };

```

DSU

```

1 struct DSU {
2     int n;
3     vector<int> p, siz;

```

```

4 DSU(int n) : n(n), p(n), siz(n,1) { iota(p.begin(),p.end(),0); }
5 int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); }
6 bool unite(int a, int b) { // false if already in the same set
7     a = find(a); b = find(b);
8     if (a == b) return false;
9     if (siz[a] < siz[b]) swap(a,b);
10    p[b] = a; siz[a] += siz[b];
11    return true;
12 }
13 bool same(int a, int b) { return find(a) == find(b); }
14 int size(int x) { return siz[find(x)]; }
15 };

```

Fenwick Tree 2D Range

```

1 struct Fenwick2DRange {
2     int n, m;
3     vector<vector<ll>>> b[4];
4     Fenwick2DRange(int n, int m) : n(n), m(m) {
5         for (auto& b : b) b.assign(n+1,vector<ll>(m+1));
6     }
7     void add(int x, int y, ll v) { // point update on the difference array
8         ll c[4] = {v, v*x, v*y, v*x*y};
9         for (int i = x+1; i <= n; i += i&-i)
10            for (int j = y+1; j <= m; j += j&-j)
11                for (int k = 0; k < 4; k++) b[k][i][j] += c[k];
12    }
13    void update(int x1, int y1, int x2, int y2, ll v) { // [x1, x2) x [y1, y2) += v
14        add(x1,y1,v); add(x1,y2,-v); add(x2,y1,-v); add(x2,y2,v);
15    }
16    ll pre(int x, int y) { // sum of [0, x) x [0, y)
17        ll s[4] = {};
18        for (int i = x; i > 0; i -= i&-i)
19            for (int j = y; j > 0; j -= j&-j)
20                for (int k = 0; k < 4; k++) s[k] += b[k][i][j];
21        return s[0]*x*y - s[1]*y - s[2]*x + s[3];
22    }
23    ll query(int x1, int y1, int x2, int y2) { // [x1, x2) x [y1, y2)
24        return pre(x2,y2) - pre(x1,y2) - pre(x2,y1) + pre(x1,y1);
25    }
26 };

```

Fenwick Tree 2D

```

1 struct Fenwick2D {
2     int n, m;
3     vector<vector<ll>>> t;
4     Fenwick2D(int n, int m) : n(n), m(m), t(n+1,vector<ll>(m+1)) {}
5     Fenwick2D(const vector<vector<ll>>& a) : Fenwick2D(a.size(),a[0].size()) {
6         for (int i = 0; i < n; i++) for (int j = 0; j < m; j++) add(i,j,a[i][j]);
7     }
8     void add(int x, int y, ll v) {
9         for (int i = x+1; i <= n; i += i&-i)
10            for (int j = y+1; j <= m; j += j&-j) t[i][j] += v;
11    }
12    ll pre(int x, int y) { // sum of [0, x) x [0, y)
13        ll s = 0;
14        for (int i = x; i > 0; i -= i&-i)
15            for (int j = y; j > 0; j -= j&-j) s += t[i][j];
16        return s;
17    }
18    ll query(int x1, int y1, int x2, int y2) { // [x1, x2) x [y1, y2)
19        return pre(x2,y2) - pre(x1,y2) - pre(x2,y1) + pre(x1,y1);
20    }
21    void set(int x, int y, ll v) { add(x,y,v - query(x,y,x+1,y+1)); }
22 };

```

Fenwick Tree Range

```

1 struct FenwickRange {
2     int n;
3     vector<ll> b1, b2;
4     FenwickRange(int n) : n(n), b1(n+1), b2(n+1) {}
5     void add(vector<ll>& b, int i, ll v) { for (i++; i <= n; i += i&-i) b[i] += v; }
6     ll sum(vector<ll>& b, int i) { ll s = 0; for (; i > 0; i -= i&-i) s += b[i]; return s; }
7     void update(int l, int r, ll v) { // a[l..r-1] += v
8         add(b1,l,v); add(b1,r,-v);
9         add(b2,l,v*1); add(b2,r,-v*r);
10    }
11    ll pre(int i) { return sum(b1,i)*i - sum(b2,i); } // sum of [0, i)
12    ll query(int l, int r) { return pre(r) - pre(l); } // [l, r)
13 };

```

Fenwick Tree

```
1 struct Fenwick {
2     int n;
3     vector<ll> t;
4     Fenwick(int n) : n(n), t(n+1) {}
5     Fenwick(const vector<ll>& a) : n(a.size()), t(n+1) { // O(n)
6         for (int i = 1; i <= n; i++) {
7             t[i] += a[i-1];
8             int j = i + (i&-i);
9             if (j <= n) t[j] += t[i];
10        }
11    }
12    void add(int i, ll v) { for (i++; i <= n; i += i&-i) t[i] += v; }
13    ll pre(int i) { // sum of [0, i)
14        ll s = 0;
15        for (; i > 0; i -= i&-i) s += t[i];
16        return s;
17    }
18    ll query(int l, int r) { return pre(r) - pre(l); } // [l, r)
19    void set(int i, ll v) { add(i, v - query(i,i+1)); }
20    int lower_bound(ll v) { // first i with pre(i+1) >= v, (values >= 0), n if none
21        int pos = 0;
22        for (int k = 1<<__lg(n); k; k >>= 1)
23            if (pos+k <= n && t[pos+k] < v) { pos += k; v -= t[pos]; }
24        return pos;
25    }
26 };
```

HLD

```
1 struct HLDNode { ll val = 0; };
2 struct HLDTree {
3     int n;
4     vector<HLDNode> t;
5     vector<ll> lazy;
6     HLDTree(int n) : n(n), t(4*n), lazy(4*n) {}
7     HLDNode merge(const HLDNode& a, const HLDNode& b) { return {a.val + b.val}; }
8     void apply(int x, int l, int r, ll v) { t[x].val += v * (r-l); lazy[x] += v; }
9     void push(int x, int l, int r) {
10         if (!lazy[x] || r-l == 1) return;
11         int m=(l+r)/2; apply(2*x+1,l,m,lazy[x]); apply(2*x+2,m,r,lazy[x]); lazy[x]=0;
12     }
13     void build(const vector<ll>& a, int x, int l, int r) {
14         if (r-l==1) { t[x].val=a[l]; return; }
15         int m=(l+r)/2; build(a,2*x+1,l,m); build(a,2*x+2,m,r); t[x]=merge(t[2*x+1],t[2*x+2]);
16     }
17     void build(const vector<ll>& a) { build(a,0,0,n); }
18     void update(int ql,int qr,ll v,int x,int l,int r) {
19         if (qr<=l || r<=ql) return;
20         if (ql<=l && r<=qr) { apply(x,l,r,v); return; }
21         push(x,l,r); int m=(l+r)/2; update(ql,qr,v,2*x+1,l,m); update(ql,qr,v,2*x+2,m,r); t[x]=merge(t[2*x+1],t[2*x+2]);
22     }
23     void update(int l,int r,ll v) { update(l,r,v,0,0,n); }
24     HLDNode query(int ql,int qr,int x,int l,int r) {
25         if (ql<=l && r<=qr) return t[x];
26         push(x,l,r); int m=(l+r)/2;
27         if (qr<=m) return query(ql,qr,2*x+1,l,m);
28         if (ql>=m) return query(ql,qr,2*x+2,m,r);
29         return merge(query(ql,qr,2*x+1,l,m),query(ql,qr,2*x+2,m,r));
30     }
31     HLDNode query(int l,int r) { return query(l,r,0,0,n); }
32 };
33
34 struct HLD {
35     int n;
36     vector<vector<int>>> g;
37     vector<int> par, dep, siz, heavy, head, pos;
38     HLDTree st;
39     HLD(int n) : n(n), g(n), par(n,-1), dep(n), siz(n,1), heavy(n,-1), head(n), pos(n), st(n) {}
40     void add_edge(int a, int b) { g[a].push_back(b); g[b].push_back(a); }
41     void build(const vector<ll>& a, int root = 0) {
42         vector<int> ord, s = {root};
43         while (!s.empty()) { // preorder, sets parent and depth
44             int u = s.back(); s.pop_back(); ord.push_back(u);
45             for (int v : g[u]) if (v != par[u]) { par[v] = u; dep[v] = dep[u]+1; s.push_back(v); }
46         }
47         for (int i = n-1; i > 0; i--) siz[par[ord[i]]] += siz[ord[i]];
48         for (int i = n-1; i > 0; i--) {
49             int u = ord[i], p = par[u];
50             if (heavy[p] < 0 || siz[u] > siz[heavy[p]]) heavy[p] = u;
```

```

51 }
52 int cur = 0; head[root] = root; s = {root};
53 while (!s.empty()) { // heavy child is pushed last, so it is visited right after u
54     int u = s.back(); s.pop_back(); pos[u] = cur++;
55     for (int v : g[u]) if (v != par[u] && v != heavy[u]) { head[v] = v; s.push_back(v); }
56     if (heavy[u] >= 0) { head[heavy[u]] = head[u]; s.push_back(heavy[u]); }
57 }
58 vector<ll> b(n);
59 for (int i = 0; i < n; i++) b[pos[i]] = a[i];
60 st.build(b);
61 }
62 void update_path(int a, int b, ll v) { // vertices on the path a-b
63     for (; head[a] != head[b]; a = par[head[a]]) {
64         if (dep[head[a]] < dep[head[b]]) swap(a,b);
65         st.update(pos[head[a]], pos[a]+1, v);
66     }
67     if (dep[a] > dep[b]) swap(a,b);
68     st.update(pos[a], pos[b]+1, v);
69 }
70 HLDNode query_path(int a, int b) {
71     HLDNode res; // identity of merge
72     for (; head[a] != head[b]; a = par[head[a]]) {
73         if (dep[head[a]] < dep[head[b]]) swap(a,b);
74         res = st.merge(res, st.query(pos[head[a]], pos[a]+1));
75     }
76     if (dep[a] > dep[b]) swap(a,b);
77     return st.merge(res, st.query(pos[a], pos[b]+1));
78 }
79 void update_subtree(int u, ll v) { st.update(pos[u], pos[u]+siz[u], v); }
80 HLDNode query_subtree(int u) { return st.query(pos[u], pos[u]+siz[u]); }
81 int lca(int a, int b) {
82     for (; head[a] != head[b]; a = par[head[a]])
83         if (dep[head[a]] < dep[head[b]]) swap(a,b);
84     return dep[a] < dep[b] ? a : b;
85 }
86 };
87
88 // Ejemplo de uso anterior (main comentado para compilar con la plantilla):
89 // int main() {
90 //     int n = 5;
91 //     HLD h(n);
92 //     h.add_edge(0,1); h.add_edge(0,2); h.add_edge(1,3); h.add_edge(1,4);
93 //     h.build({1,2,3,4,5});
94 //     h.update_path(3,2,10);
95 //     cout << h.query_path(4,2).val << "\n";
96 //     cout << h.query_subtree(1).val << "\n";
97 // }

```

Ordered Set

```

1 #include <bits/stdc++.h>
2 #include <ext/pb_ds/assoc_container.hpp>
3 #include <ext/pb_ds/tree_policy.hpp>
4 using namespace std;
5 using namespace __gnu_pbds;
6
7 template <class T>
8 using ordered_set = tree<
9 T,
10 null_type,
11 less<T>, // use less_equal<T> if you want to allow duplicates (see note below)
12 rb_tree_tag,
13 tree_order_statistics_node_update>;
14
15 // --- Example usage ---
16
17 // Ejemplo de uso anterior (comentado para pegar el algoritmo con la plantilla):
18 // int main() {
19 //     ios::sync_with_stdio(false);
20 //     cin.tie(nullptr);
21 //
22 //     ordered_set<int> s;
23 //
24 //     s.insert(10);
25 //     s.insert(20);
26 //     s.insert(30);
27 //     s.insert(40);
28 //
29 //     // [1] Get element by index (0-based)
30 //     cout << "Element at index 2: " << *s.find_by_order(2) << "\n"; // -> 30
31 //
32 //     // [2] Get index of an element
33 //     cout << "Index of 30: " << s.order_of_key(30) << "\n"; // -> 2

```

```

34 //
35 // // [3] Lower bound behavior
36 // cout << "Index where 25 would be inserted: " << s.order_of_key(25) << "\n"; // -> 2
37 //
38 // // [4] Iterate through all elements
39 // for (auto x : s) cout << x << " ";
40 // cout << "\n";
41 //
42 // // [5] Check if an element exists
43 // if (s.find(20) != s.end())
44 //     cout << "20 exists in set\n";
45 // }

```

Persistent Segment Tree

```

1 struct Node {
2     ll val = 0;
3     int lc = 0, rc = 0; // children, 0 = null node
4 };
5 struct PerSegTree {
6     int n;
7     vector<Node> t;
8     vector<int> root; // root[i] = root node of version i
9     PerSegTree(int n, int cap = 0) : n(n), t(1), root(1,0) { t.reserve(cap+1); }
10    Node merge(const Node& a, const Node& b) {
11        Node res;
12        res.val = a.val + b.val; // your merge here
13        return res;
14    }
15    int build(const vector<ll>& a, int l, int r) {
16        int x = t.size(); t.emplace_back();
17        if (r - l == 1) {t[x].val = a[l]; return x; }
18        int m = (l+r)/2;
19        int lc = build(a,l,m), rc = build(a,m,r);
20        t[x] = merge(t[lc],t[rc]); t[x].lc = lc; t[x].rc = rc;
21        return x;
22    }
23    int update(int p, int i, ll v, int l, int r) { // returns the new node
24        int x = t.size(); t.push_back(t[p]);
25        if (r - l == 1) {t[x].val += v; return x; } // your apply here (= v to assign)
26        int m = (l+r)/2;
27        if (i < m) {int c = update(t[p].lc,i,v,l,m); t[x].lc = c; }
28        else {int c = update(t[p].rc,i,v,m,r); t[x].rc = c; }
29        int lc = t[x].lc, rc = t[x].rc;
30        t[x] = merge(t[lc],t[rc]); t[x].lc = lc; t[x].rc = rc;
31        return x;
32    }
33    Node query(int ql, int qr, int x, int l, int r) {
34        if (ql <= l && r <= qr) return t[x];
35        int m = (l + r) / 2;
36        if (qr <= m) return query(ql,qr,t[x].lc,l,m);
37        if (ql >= m) return query(ql,qr,t[x].rc,m,r);
38        return merge(query(ql,qr,t[x].lc,l,m),
39                      query(ql,qr,t[x].rc,m,r));
40    }
41    int kth(int a, int b, ll k) { // k-th smallest (0 idx) in the count difference of roots b and a
42        int l = 0, r = n;
43        while (r - l > 1) {
44            int m = (l+r)/2;
45            ll c = t[t[b].lc].val - t[t[a].lc].val;
46            if (k < c) {a = t[a].lc; b = t[b].lc; r = m; }
47            else {k -= c; a = t[a].rc; b = t[b].rc; l = m; }
48        }
49        return l;
50    }
51    void build(const vector<ll>& a){ root[0] = build(a,0,n); }
52    int update(int ver, int i, ll v){ root.push_back(update(root[ver],i,v,0,n)); return root.size()-1; } // new version id
53    Node query(int ver, int l, int r){ return query(l,r,root[ver],0,n); }
54 };
55
56 // Ejemplo de uso (main comentado para compilar con la plantilla):
57 // int main() {
58 //     vector<ll> values = {2, 1, 3, 4};
59 //     PerSegTree pst(values.size());
60 //     pst.build(values); // version 0
61 //     int version1 = pst.update(0, 1, 5); // new version, index 1 += 5
62 //     cout << pst.query(0, 0, 4).val << "\n";
63 //     cout << pst.query(version1, 0, 4).val << "\n";
64 // }

```

Segment Tree Lazy

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 using ll = long long;
4 struct Node {
5     ll val = 0;
6 };
7 struct SegTree {
8     int n;
9     vector<Node> t;
10    vector<ll> lz;
11    SegTree(int n) : n(n), t(4*n), lz(4*n) {}
12    Node merge(const Node& a, const Node& b) {
13        Node res;
14        res.val = a.val + b.val;
15        return res;
16    }
17    void apply(int x, int l, int r, ll v) {
18        t[x].val += v*(r-l);
19        lz[x] += v;
20    }
21    void push(int x, int l, int r) {
22        if (!lz[x]) return;
23        int m = (l+r)/2;
24        apply(2*x+1, l, m, lz[x]);
25        apply(2*x+2, m, r, lz[x]);
26        lz[x] = 0;
27    }
28    void build(const vector<ll>& a, int x, int l, int r) {
29        if (r - l == 1) {t[x].val = a[l]; return;}
30        int m = (l+r)/2;
31        build(a, 2*x+1, l, m);
32        build(a, 2*x+2, m, r);
33        t[x] = merge(t[2*x+1], t[2*x+2]);
34    }
35    void update(int ql, int qr, ll v, int x, int l, int r) {
36        if (qr <= l || r <= ql) return;
37        if (ql <= l && r <= qr) {apply(x, l, r, v); return;}
38        push(x, l, r);
39        int m = (l+r)/2;
40        update(ql, qr, v, 2*x+1, l, m);
41        update(ql, qr, v, 2*x+2, m, r);
42        t[x] = merge(t[2*x+1], t[2*x+2]);
43    }
44    Node query(int ql, int qr, int x, int l, int r) {
45        if (ql <= l && r <= qr) return t[x];
46        push(x, l, r);
47        int m = (l + r) / 2;
48        if (qr <= m) return query(ql, qr, 2*x+1, l, m);
49        if (ql >= m) return query(ql, qr, 2*x+2, m, r);
50        return merge(query(ql, qr, 2*x+1, l, m),
51                      query(ql, qr, 2*x+2, m, r));
52    }
53    void build(const vector<ll>& a) { build(a, 0, 0, n); }
54    void update(int l, int r, ll v) { update(l, r, v, 0, 0, n); }
55    Node query(int l, int r) { return query(l, r, 0, 0, n); }
56 };
```

Segment Tree

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 using ll = long long;
4 struct Node {
5     ll val = 0;
6 };
7 struct SegTree {
8     int n;
9     vector<Node> t;
10    SegTree(int n) : n(n), t(4*n) {}
11    Node merge(const Node& a, const Node& b) {
12        Node res;
13        return res;
14    }
15    void update(int i, ll v, int x, int l, int r) {
16        if (r - l == 1) {t[x].val = v; return;}
17        int m = (l+r)/2;
18        if (i < m) update(i, v, 2*x+1, l, m);
19        else update(i, v, 2*x+2, m, r);
20        t[x] = merge(t[2*x+1], t[2*x+2]);
21    }
```

```

22 Node query(int ql, int qr, int x, int l, int r) {
23     if (ql <= l && r <= qr) return t[x];
24     int m = (l + r) / 2;
25     if (qr <= m) return query(ql,qr,2*x+1,l,m);
26     if (ql >= m) return query(ql,qr,2*x+2,m,r);
27     return merge(query(ql,qr,2*x+1,l,m),
28                 query(ql,qr,2*x+2,m,r));
29 }
30 void update(int i, ll v){ update(i,v, 0,0,n); }
31 Node query(int l, int r){ return query(l,r,0,0,n); }
32 };

```

Sparse Table

```

1 struct SparseTable {
2     int n;
3     vector<vector<ll>> t;
4     ll merge(ll a, ll b) {
5         return min(a,b);
6     }
7     SparseTable(const vector<ll>& a) : n(a.size()) {
8         int K = __lg(n) + 1;
9         t.assign(K, vector<ll>(n));
10        t[0] = a;
11        for (int k=1; k<K; k++)
12            for (int i = 0; i+(1<<k) <= n; i++)
13                t[k][i] = merge(t[k-1][i], t[k-1][i+(1<<(k-1))]);
14    }
15    ll query(int l, int r) { // [l, r), ops (min, max, gcd, and, or)
16        int k = __lg(r-l);
17        return merge(t[k][l], t[k][r-(1<<k)]);
18    }
19    ll query_log(int l, int r) { // [l, r), any associative op
20        ll res = 1e18; // identity (0 for sum/xor)
21        for (int k = __lg(r-l); k >= 0; k--)
22            if (l + (1<<k) <= r) { res = merge(res, t[k][l]); l += 1<<k; }
23        return res;
24    }
25 };

```

Two Pointers

```

1 // Two Pointers & Sliding Window Aggregation (SWAG)
2
3 // 1. Sliding Window Estandar - O(N)
4 // Ejemplo: Maxima longitud de subarray con condicion monotona
5 int slidingWindow(const vi& a, int k) {
6     int n = sz(a), ans = 0, l = 0;
7     // Estado de la ventana (ej. frecuencias, suma)
8     // vi freq(MAX_VAL, 0); int distinct = 0;
9
10    rep(r, n) {
11        // add(a[r]);
12        // while (!valid()) { remove(a[l++]); }
13        ckmax(ans, r - l + 1);
14    }
15    return ans;
16 }
17
18 // 2. SWAG (Sliding Window Aggregation) - O(1) amortizado por push/pop/query
19 // Permite operaciones asociativas NO reversibles (ej: min, max, gcd, matrix mul)
20 template<class T>
21 struct SWAG {
22     struct Node {
23         T val, agg;
24     };
25     stack<Node> s1, s2;
26
27     T op(T a, T b) { return min(a, b); } // Cambiar operacion (gcd, max, +, etc.)
28
29     bool empty() { return s1.empty() && s2.empty(); }
30     int size() { return sz(s1) + sz(s2); }
31
32     void push(T val) {
33         T agg = s2.empty() ? val : op(s2.top().agg, val);
34         s2.push({val, agg});
35     }
36
37     void pop() {
38         if (s1.empty()) {
39             while (!s2.empty()) {
40                 T val = s2.top().val;

```

```

41         s2.pop();
42         T agg = s1.empty() ? val : op(val, s1.top().agg);
43         s1.push({val, agg});
44     }
45 }
46 s1.pop();
47 }
48
49 T query() {
50     if (empty()) return 0; // valor neutro
51     if (s1.empty()) return s2.top().agg;
52     if (s2.empty()) return s1.top().agg;
53     return op(s1.top().agg, s2.top().agg);
54 }
55 };

```

Geometry

Convex Hull

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 struct point {
6     double x, y;
7     point() { x = y = 0; }
8     point(double _x, double _y) : x(_x), y(_y) {}
9     bool operator == (point const& t) const {
10         return x == t.x && y == t.y;
11     }
12 };
13
14 int orientation(point a, point b, point c) {
15     double v = a.x*(b.y-c.y)+b.x*(c.y-a.y)+c.x*(a.y-b.y);
16     if (v < 0) return -1; // clockwise
17     if (v > 0) return +1; // counter-clockwise
18     return 0;
19 }
20
21 bool cw(point a, point b, point c, bool include_collinear) {
22     int o = orientation(a, b, c);
23     return o < 0 || (include_collinear && o == 0);
24 }
25 bool collinear(point a, point b, point c) { return orientation(a, b, c) == 0; }
26
27 void convex_hull(vector<point>& a, bool include_collinear = false) {
28     point p0 = *min_element(a.begin(), a.end(), [](point a, point b) {
29         return make_pair(a.y, a.x) < make_pair(b.y, b.x);
30     });
31     sort(a.begin(), a.end(), [&p0](const point& a, const point& b) {
32         int o = orientation(p0, a, b);
33         if (o == 0)
34             return (p0.x-a.x)*(p0.x-a.x) + (p0.y-a.y)*(p0.y-a.y)
35                 < (p0.x-b.x)*(p0.x-b.x) + (p0.y-b.y)*(p0.y-b.y);
36         return o < 0;
37     });
38     if (include_collinear) {
39         int i = (int)a.size()-1;
40         while (i >= 0 && collinear(p0, a[i], a.back())) i--;
41         reverse(a.begin()+i+1, a.end());
42     }
43
44     vector<point> st;
45     for (int i = 0; i < (int)a.size(); i++) {
46         while (st.size() > 1 && !cw(st[st.size()-2], st.back(), a[i], include_collinear))
47             st.pop_back();
48         st.push_back(a[i]);
49     }
50
51     if (include_collinear == false && st.size() == 2 && st[0] == st[1])
52         st.pop_back();
53
54     a = st;
55 }
56
57 double area(const vector<point>& fig) {
58     double res = 0;
59     for (unsigned i = 0; i < fig.size(); i++) {
60         point p = i ? fig[i-1] : fig.back();
61         point q = fig[i];

```

```

62     res += (p.x - q.x) * (p.y + q.y);
63 }
64 return fabs(res) / 2;
65 }
66
67 // example use case
68
69 // Ejemplo de uso anterior (comentado para pegar el algoritmo con la plantilla):
70 // void solve() {
71 //     double n,r;
72 //     cin >> n >> r;
73 //
74 //     vector<point> coords(n);
75 //     for(int i = 0; i < n; i++) {
76 //         cin >> coords[i].x >> coords[i].y;
77 //     }
78 //
79 //     auto dist = [](point &a, point&b) {
80 //         return sqrt(pow(a.x-b.x,2)+pow(a.y-b.y,2));
81 //     };
82 //
83 //     const double PI = 3.141516;
84 //     double cir = 2 * PI * r;
85 //     convex_hull(coords);
86 //
87 //     double per = 0;
88 //     coords.push_back(coords[0]);
89 //     for(int i = 1; i < coords.size(); i++) {
90 //         per += dist(coords[i-1],coords[i]);
91 //     }
92 //
93 //     int ans = round(per+cir);
94 //     cout << ans << endl;
95 // }
96 //
97 //
98 // int main() {
99 //     ios::sync_with_stdio(false);
100 //     cin.tie(0);
101 //     int t = 1;
102 //     //cin >> t;
103 //     while(t--)
104 //         solve();
105 // }

```

Graphs

2-SAT

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 #define fst first
6 #define snd second
7 #define ll long long
8 #define endl '\n'
9
10 const int INF = 1 << 30;
11 const int MAXN = 1e5+5;
12
13 struct TwoSatSolver {
14     int n_vars;
15     int n_vertices;
16     vector<vector<int>> adj, adj_t;
17     vector<bool> used;
18     vector<int> order, comp;
19     vector<bool> assignment;
20
21     TwoSatSolver(int _n_vars) : n_vars(_n_vars), n_vertices(2 * n_vars), adj(n_vertices), adj_t(
        n_vertices), used(n_vertices), order(), comp(n_vertices, -1), assignment(n_vars) {
22         order.reserve(n_vertices);
23     }
24     void dfs1(int v) {
25         used[v] = true;
26         for (int u : adj[v]) {
27             if (!used[u])
28                 dfs1(u);
29         }
30         order.push_back(v);
31     }

```

```

32
33 void dfs2(int v, int cl) {
34     comp[v] = cl;
35     for (int u : adj_t[v]) {
36         if (comp[u] == -1)
37             dfs2(u, cl);
38     }
39 }
40
41 bool solve_2SAT() {
42     order.clear();
43     used.assign(n_vertices, false);
44     for (int i = 0; i < n_vertices; ++i) {
45         if (!used[i])
46             dfs1(i);
47     }
48
49     comp.assign(n_vertices, -1);
50     for (int i = 0, j = 0; i < n_vertices; ++i) {
51         int v = order[n_vertices - i - 1];
52         if (comp[v] == -1)
53             dfs2(v, j++);
54     }
55
56     assignment.assign(n_vars, false);
57     for (int i = 0; i < n_vertices; i += 2) {
58         if (comp[i] == comp[i + 1])
59             return false;
60         assignment[i / 2] = comp[i] > comp[i + 1];
61     }
62     return true;
63 }
64
65 void add_disjunction(int a, bool na, int b, bool nb) {
66     // na and nb signify whether a and b are to be negated
67     a = 2 * a ^ na;
68     b = 2 * b ^ nb;
69     int neg_a = a ^ 1;
70     int neg_b = b ^ 1;
71     adj[neg_a].push_back(b);
72     adj[neg_b].push_back(a);
73     adj_t[b].push_back(neg_a);
74     adj_t[a].push_back(neg_b);
75 }
76
77 };
78
79 // Ejemplo de uso anterior (comentado para pegar el algoritmo con la plantilla):
80 // void solve() {
81 //     int n,m;
82 //     cin >> n >> m;
83 //
84 //     TwoSatSolver solver(m);
85 //     for(int i = 0; i < n; i++) {
86 //         int topping1, topping2;
87 //         char op1, op2;
88 //         cin >> op1 >> topping1 >> op2 >> topping2;
89 //         bool no_good1 = op1 == '-';
90 //         bool no_good2 = op2 == '-';
91 //         topping1--, topping2--;
92 //         solver.add_disjunction(topping1, no_good1, topping2, no_good2);
93 //     }
94 //
95 //     bool poss = solver.solve_2SAT();
96 //     if(!poss) {
97 //         cout << "IMPOSSIBLE" << endl;
98 //         return;
99 //     }
100 //
101 //     for(int i = 0; i < m; i++) {
102 //         if(solver.assignment[i]) {
103 //             cout << "+ ";
104 //         } else {
105 //             cout << "- ";
106 //         }
107 //     }
108 // }
109 //
110 // int main() {
111 //     ios::sync_with_stdio(false);
112 //     cin.tie(nullptr);
113 //     int t = 1;
114 //     //cin >> t;

```

```

115 // while(t--)
116 //     solve();
117 // }

```

Bellman Ford

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 #define fst first
6 #define snd second
7 #define ll long long
8
9 const ll INF = LONG_LONG_MAX;
10 const int MAXN = 1e5+5;
11
12 struct Edge {
13     int u, v;
14     ll weight;
15 };
16
17 int n,m;
18 vector<Edge> edges;
19
20 // Bellman-Ford algorithm to detect negative cycles and find shortest paths
21 bool bellman(int src, vector<ll> &dist) {
22     dist.assign(n + 1, INF);
23     dist[src] = 0;
24     for(int i = 2; i <= n; i++)
25     {
26         for (int j = 0; j < m; j++)
27         {
28             int u = edges[j].u;
29             int v = edges[j].v;
30             ll weight = edges[j].weight;
31             if (dist[u]!=INF && dist[u] + weight < dist[v])
32                 dist[v] = dist[u] + weight;
33         }
34     }
35     for (int i = 0; i < m; i++)
36     {
37         int u = edges[i].u;
38         int v = edges[i].v;
39         ll weight = edges[i].weight;
40         // True if neg-cylce exists
41         if (dist[u]!=INF && dist[u] + weight < dist[v])
42             return true;
43     }
44     return false;
45 }
46
47 // Ejemplo de uso anterior (comentado para pegar el algoritmo con la plantilla):
48 // int main() {
49 //     cin >> n >> m;
50 //     edges.resize(m);
51 //
52 //     for (int i = 0; i < m; i++) {
53 //         int a, b;
54 //         ll w;
55 //         cin >> a >> b >> w;
56 //         edges[i] = {a, b, w};
57 //     }
58 //
59 //     vector<ll> dist;
60 //
61 //     bool hasNegativeCycle = bellman(1, dist);
62 //
63 //     if (hasNegativeCycle) {
64 //         cout << "negative-weight cycle\n";
65 //     } else {
66 //         if (dist[n] == INF) {
67 //             cout << "no path from 1 to " << n << "\n";
68 //         } else {
69 //             cout << "shortest path from 1 to " << n << " is: " << dist[n] << "\n";
70 //         }
71 //     }
72 // }

```

Cd

```

1 #include "bits/stdc++.h"

```

```

2 using namespace std;
3
4 using ll = long long;
5 using ld = long double;
6 #define sz(v) ((int)((v).size()))
7
8 const int MAXN = 100005;
9
10 int n;
11 vector<int> adj[MAXN];
12 int sub[MAXN];
13 bool removed[MAXN];
14 int parCentroid[MAXN];
15 vector<vector<pair<int, int>>> elements(MAXN);
16
17 vector<vector<pair<int, int>>> sufix(MAXN);
18 #define linea() cerr << "-----" << '\n'
19 int dfs_size(int u, int p) {
20     sub[u] = 1;
21     for (int v : adj[u]) {
22         if (v == p || removed[v]) continue;
23         sub[u] += dfs_size(v, u);
24     }
25     return sub[u];
26 }
27
28 int find_centroid(int u, int p, int total) {
29     for (int v : adj[u]) {
30         if (v == p || removed[v]) continue;
31         if (sub[v] > total / 2)
32             return find_centroid(v, u, total);
33     }
34     return u;
35 }
36 void bfs(int node) {
37     map<int, int> dist;
38     queue<int> q;
39     q.push(node);
40     elements[node].push_back({node, 0});
41     while (!q.empty()) {
42         auto l = q.front();
43         q.pop();
44         for (auto& x : adj[l]) {
45             if (dist[x] == 0 && !removed[x] && x != node) {
46                 dist[x] = dist[l] + 1;
47                 q.push(x);
48                 elements[node].push_back({x, dist[x]});
49             }
50         }
51     }
52
53     sort(elements[node].begin(), elements[node].end());
54
55     sufix[node].resize(sz(elements[node]) + 1);
56
57     pair<int, int> otro {1e9, 1e9};
58     for (int i = sz(elements[node]) - 1; i >= 0; i--) {
59         if (otro.second > elements[node][i].second) {
60             otro = elements[node][i];
61         }
62         else if (otro.second == elements[node][i].second) {
63             if (otro.first > elements[node][i].first) otro = elements[node][i];
64         }
65         sufix[node][i] = otro;
66     }
67 }
68 void decompose(int u, int p = -1) {
69     int total = dfs_size(u, -1);
70     int c = find_centroid(u, -1, total);
71     bfs(c);
72     removed[c] = true;
73
74     parCentroid[c] = (p == -1 ? c : p); // root centroid points to itself
75
76     // Example: hook custom logic here (before recursing)
77     // e.g., preprocess distances from c to other nodes in its component
78
79     for (int v : adj[c]) {
80         if (!removed[v])
81             decompose(v, c);
82     }
83 }
84 int bs(vector<pair<int, int>>& v, int target) {

```

```

85     int low = 0, high = sz(v) - 1;
86     int ans = -1;
87     while (low <= high) {
88         int mid = low + (high - low) / 2;
89         if (v[mid].first > target) {
90             ans = mid;
91             high = mid - 1;
92         }
93         else low = mid + 1;
94     }
95     return ans;
96 }
97 int dist(vector<pair<int, int>>& v, int target) {
98     int low = 0, high = sz(v) - 1;
99     int ans = -1;
100    while (low <= high) {
101        int mid = low + (high - low) / 2;
102        if (v[mid].first == target) {
103            return mid;
104        }
105        else if (v[mid].first < target) low = mid + 1;
106        else high = mid - 1;
107    }
108    return -1;
109 }
110
111 // Ejemplo de uso anterior (comentado para compilar con la plantilla):
112 // int32_t main() {
113 //     ios::sync_with_stdio(false);
114 //     cin.tie(nullptr);
115 //     cin >> n;
116 //     for (int i = 0; i < n - 1; i++) {
117 //         int u, v;
118 //         cin >> u >> v;
119 //         adj[u].push_back(v);
120 //         adj[v].push_back(u);
121 //     }
122 //
123 //     decompose(1);
124 //     for (int i = 1; i < n; i++) {
125 //         int aux = i;
126 //         int ori = aux;
127 //         pair<int, int> res = {1e9, 1e9};
128 //         bool ya = false;
129 //         do {
130 //             int it = bs(elements[aux], ori);
131 //             if (it == -1) {
132 //                 aux = parCentroid[aux];
133 //                 linea();
134 //                 continue;
135 //             }
136 //             pair<int, int> ans = sufix[aux][it];
137 //             int it2 = dist(elements[aux], ori);
138 //             ans.second += elements[aux][it2].second;
139 //             if (ans.second < res.second) {
140 //                 res = ans;
141 //             }
142 //             else if (ans.second == res.second) {
143 //                 if (ans.first < res.first) {
144 //                     res = ans;
145 //                 }
146 //             }
147 //             aux = parCentroid[aux];
148 //             linea();
149 //             if (ya) break;
150 //             if (parCentroid[aux] == aux) ya = true;
151 //         } while (true);
152 //         cout << res.first << ' ';
153 //     }
154 //     cout << n << '\n';
155 //
156 //     return 0;
157 // }

```

Centroid Decomposition

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 #define endl '\n'
5 using ll = long long;
6
7 const int MAXN = 200005;

```

```

8
9 // ----- GLOBALS -----
10 int n;
11 vector<int> adj[MAXN];
12 int sub[MAXN]; // subtree sizes
13 bool removed[MAXN]; // centroid removed flag
14 int parCentroid[MAXN]; // parent in centroid tree
15
16 int dfs_size(int u, int p) {
17     sub[u] = 1;
18     for (int v : adj[u]) {
19         if (v == p || removed[v]) continue;
20         sub[u] += dfs_size(v, u);
21     }
22     return sub[u];
23 }
24
25 int find_centroid(int u, int p, int total) {
26     for (int v : adj[u]) {
27         if (v == p || removed[v]) continue;
28         if (sub[v] > total / 2)
29             return find_centroid(v, u, total);
30     }
31     return u;
32 }
33
34 void decompose(int u, int p = -1) {
35     int total = dfs_size(u, -1);
36     int c = find_centroid(u, -1, total);
37     removed[c] = true;
38
39     parCentroid[c] = (p == -1 ? c : p); // root centroid points to itself
40
41     // Example: hook custom logic here (before recursing)
42     // e.g., preprocess distances from c to other nodes in its component
43
44     for (int v : adj[c]) {
45         if (!removed[v])
46             decompose(v, c);
47     }
48 }
49
50 // ----- MAIN -----
51
52 // Ejemplo de uso anterior (comentado para pegar el algoritmo con la plantilla):
53 // int main() {
54 //     cin >> n;
55 //     for (int i = 0; i < n - 1; i++) {
56 //         int u, v;
57 //         cin >> u >> v;
58 //         adj[u].push_back(v);
59 //         adj[v].push_back(u);
60 //     }
61 //
62 //     decompose(0);
63 //     cout << "Centroid decomposition parent array:\n";
64 //     for (int i = 0; i < n; i++)
65 //         cout << i << " -> " << parCentroid[i] << endl;
66 //
67 //     return 0;
68 // }

```

Dijkstra

```

1 const ll INF = LLONG_MAX / 4;
2 struct Dijkstra {
3     int n;
4     vector<vector<pair<int,ll>>> g;
5     vector<ll> d;
6     vector<int> par;
7     Dijkstra(int n) : n(n), g(n) {}
8     void add_edge(int a, int b, ll w) { g[a].push_back({b,w}); } // directed
9     void add_undirected(int a, int b, ll w) { add_edge(a,b,w); add_edge(b,a,w); }
10    void run(const vector<int>& src) { // multi-source
11        d.assign(n,INF); par.assign(n,-1);
12        priority_queue<pair<ll,int>,vector<pair<ll,int>>,greater<>> pq;
13        for (int s : src) { d[s] = 0; pq.push({0,s}); }
14        while (!pq.empty()) {
15            auto [du,u] = pq.top(); pq.pop();
16            if (du > d[u]) continue; // stale entry
17            for (auto [v,w] : g[u])
18                if (d[u]+w < d[v]) { d[v] = d[u]+w; par[v] = u; pq.push({d[v],v}); }
19        }

```

```

20 }
21 void run(int s) { run(vector<int>{s}); }
22 vector<int> path(int t) { // s -> t, empty if unreachable
23     vector<int> p;
24     if (d[t] >= INF) return p;
25     for (; t >= 0; t = par[t]) p.push_back(t);
26     reverse(p.begin(), p.end());
27     return p;
28 }
29 };

```

Floyd Warshall

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 #define fst first
6 #define snd second
7 #define ll long long
8
9 const int INF = 1 << 30;
10 const int MAXN = 1e3+5;
11
12 int n;
13 ll d[MAXN][MAXN];
14
15 void floyd_warshall() {
16     for (int k = 1; k <= n; k++)
17         for (int i = 1; i <= n; i++)
18             for (int j = 1; j <= n; j++)
19                 if (d[i][k] + d[k][j] < d[i][j])
20                     d[i][j] = d[i][k] + d[k][j];
21 }
22
23 // Ejemplo de uso anterior (comentado para pegar el algoritmo con la plantilla):
24 // void solve() {
25 //     int s, t;
26 //     cin >> n;
27 //     cin >> s >> t;
28 //     for(int i = 1; i <= n; i++) {
29 //         for(int j = 1; j <= n; j++) {
30 //             cin >> d[i][j];
31 //             if(d[i][j] == -1) d[i][j] = INF;
32 //         }
33 //     }
34 //
35 //     if(d[s][t] == INF)
36 //         cout << "-1\n";
37 //     else
38 //         cout << d[s][t] << '\n';
39 // }
40 //
41 // int main() {
42 //     ios::sync_with_stdio(false);
43 //     cin.tie(nullptr);
44 //     int t = 1;
45 //     //cin >> t;
46 //     while(t--)
47 //         solve();
48 // }

```

Ford Fulkerson

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 #define ll long long
6 #define endl '\n'
7
8 const ll INF = 1e18;
9
10 struct Dinic {
11     struct Edge {
12         int to, rev;
13         ll cap;
14     };
15
16     int n;
17     vector<vector<Edge>> g;
18     vector<int> level, it;

```

```

19 Dinic(int n) : n(n), g(n), level(n), it(n) {}
20
21
22 void add_edge(int u, int v, ll cap) {
23     g[u].push_back({v, (int)g[v].size(), cap});
24     g[v].push_back({u, (int)g[u].size() - 1, 0});
25 }
26
27 bool bfs(int s, int t) {
28     fill(level.begin(), level.end(), -1);
29     queue<int> q;
30     level[s] = 0; q.push(s);
31     while (!q.empty()) {
32         int u = q.front(); q.pop();
33         for (auto &e : g[u]) {
34             if (e.cap > 0 && level[e.to] == -1) {
35                 level[e.to] = level[u] + 1;
36                 q.push(e.to);
37             }
38         }
39     }
40     return level[t] != -1;
41 }
42
43 ll dfs(int u, int t, ll f) {
44     if (u == t) return f;
45     for (int &i = it[u]; i < (int)g[u].size(); i++) {
46         Edge &e = g[u][i];
47         if (e.cap > 0 && level[e.to] == level[u] + 1) {
48             ll ret = dfs(e.to, t, min(f, e.cap));
49             if (ret > 0) {
50                 e.cap -= ret;
51                 g[e.to][e.rev].cap += ret;
52                 return ret;
53             }
54         }
55     }
56     return 0;
57 }
58
59 ll max_flow(int s, int t) {
60     ll flow = 0, f;
61     while (bfs(s, t)) {
62         fill(it.begin(), it.end(), 0);
63         while ((f = dfs(s, t, INF)) > 0)
64             flow += f;
65     }
66     return flow;
67 }
68
69 // Min-cut edges
70 void print_mincut_edges(int s) {
71     vector<bool> vis(n, false);
72     queue<int> q;
73     q.push(s);
74     vis[s] = true;
75     while (!q.empty()) {
76         int u = q.front(); q.pop();
77         for (auto &e : g[u]) {
78             if (e.cap > 0 && !vis[e.to]) {
79                 vis[e.to] = true;
80                 q.push(e.to);
81             }
82         }
83     }
84     for (int u = 0; u < n; ++u) {
85         if (vis[u]) {
86             for (auto &e : g[u]) {
87                 if (!vis[e.to] && e.cap == 0) {
88                     cout << u + 1 << " " << e.to + 1 << endl;
89                 }
90             }
91         }
92     }
93 }
94 };
95
96 // Ejemplo de uso anterior (comentado para pegar el algoritmo con la plantilla):
97 // int main() {
98 //     int n,m;
99 //     cin >> n >> m;
100 //     Dinic g(n);
101 //     for (int i = 0; i < m; ++i) {

```

```

102 //      int u, v, w;
103 //      cin >> u >> v;
104 //      w = 1;
105 //      u--;v--;
106 //      g.add_edge(u, v, w);
107 //  }
108 //  cout << g.max_flow(0,n-1) << endl;
109 //  g.print_mincut_edges(0);
110 // }

```

Functional Graph

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 #define endl '\n'
6
7 const int MAXN = 2e5+5;
8 const int LOG = 22;
9
10 int up[MAXN][LOG];
11 bool visited[MAXN];
12 bool on_stack[MAXN];
13 int next_node[MAXN];
14 int cycle[MAXN];
15 int cycle_length[MAXN];
16 int depth[MAXN];
17
18 int cycle_idx[MAXN];
19
20 void mark_cycle(int u, int id) {
21     cycle[u] = id;
22     depth[u] = 0;
23     cycle_length[id]++;
24     cycle_idx[u] = cycle_length[id];
25     if(!cycle[next_node[u]])
26         mark_cycle(next_node[u], id);
27 }
28
29 void dfs(int u) {
30     visited[u] = 1;
31     on_stack[u] = 1;
32
33     int v = next_node[u];
34     if(on_stack[v]) {
35         mark_cycle(v, v);
36     } else if(!visited[v]){
37         dfs(v);
38     }
39
40     if(!cycle[u])
41         depth[u] = depth[v] + 1;
42
43     on_stack[u] = 0;
44 }
45
46 int jumpto(int u, int k) {
47     int v = u;
48     for(int i = 0; i < LOG; i++) {
49         if((1 << i) & k) {
50             v = up[v][i];
51         }
52     }
53     return v;
54 }
55
56
57 int distcycle(int u, int v) {
58     if(cycle_idx[u] < cycle_idx[v]) {
59         return cycle_idx[v] - cycle_idx[u];
60     }
61     int turn = (cycle_length[cycle[u]] - cycle_idx[u] + 1) % cycle_length[cycle[u]];
62     return turn + cycle_idx[v] - 1;
63 }
64
65 // Ejemplo de uso anterior (comentado para pegar el algoritmo con la plantilla):
66 // void solve() {
67 //     int n,q;
68 //     cin >> n >> q;
69 //
70 //     for(int i = 1; i <= n; i++) {
71 //         cin >> next_node[i];

```

```

72 //     up[i][0] = next_node[i];
73 // }
74 //
75 // for(int j = 1; j < LOG; j++) {
76 //     for(int i = 1; i <= n; i++) {
77 //         up[i][j] = up[up[i][j-1]][j-1];
78 //     }
79 // }
80 //
81 // for(int i = 1; i <= n; i++) {
82 //     if(!visited[i]) dfs(i);
83 // }
84 //
85 // while(q--) {
86 //     int u,v;
87 //     cin >> u >> v;
88 //
89 //     int cyu = cycle[u];
90 //     int cyv = cycle[v];
91 //
92 //     if(cyu == 0) {
93 //         cyu = cycle[jumpton(u,depth[u])];
94 //     }
95 //     if(cyv == 0) {
96 //         cyv = cycle[jumpton(v,depth[v])];
97 //     }
98 //
99 //     int a = u;
100 //     int b = v;
101 //
102 //     if(cyu != cyv) {
103 //         cout << -1 << endl;
104 //         continue;
105 //     }
106 //
107 //     if(cycle[u] == 0 && cycle[v] == 0) {
108 //         if(depth[u] < depth[v]) {
109 //             cout << -1 << endl;
110 //             continue;
111 //         }
112 //         int diff = depth[u] - depth[v];
113 //         if(jumpton(u,diff) != v) {
114 //             cout << -1 << endl;
115 //         } else {
116 //             cout << diff << endl;
117 //         }
118 //         continue;
119 //     }
120 //
121 //     if(cycle[u] && cycle[v]) {
122 //         cout << distcycle(u,v) << endl;
123 //         continue;
124 //     }
125 //
126 //     if(cycle[u] == 0) {
127 //         int steps = depth[u];
128 //         steps += distcycle(jumpton(u,depth[u]), v);
129 //         cout << steps << endl;
130 //         continue;
131 //     }
132 //
133 //     if(cycle[v] == 0) {
134 //         cout << -1 << endl;
135 //         continue;
136 //     }
137 // }
138 // }
139 //
140 // int main() {
141 //     ios::sync_with_stdio(false);
142 //     cin.tie(nullptr);
143 //     int t = 1;
144 //     //cin >> t;
145 //     while(t--)
146 //         solve();
147 // }

```

Hungarian

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4

```

```

5 #define fst first
6 #define snd second
7 #define ll long long
8 #define ms(a, b) memset(a, b, sizeof(a))
9
10 const int INF = 1 << 30;
11 const int MAXN = 25;
12 const int MAXM = 25;
13
14 // hungarian weighted matching algo
15 // finds the MAX cost of MAX matching, to find mincost, add edges as negatives
16 // Nodes are indexed from 1 on both sides
17 template<typename T>
18 struct KuhnMunkras { // n for left, m for right
19     int n, m, match[MAXN];
20     T g[MAXN][MAXM], lx[MAXN], ly[MAXM], slack[MAXM];
21     bool vx[MAXN], vy[MAXM];
22
23     void init(int n_, int m_) {
24         ms(g,0); n = n_, m = m_;
25     }
26
27     void add(int u, int v, T w) {
28         g[u][v] = w;
29     }
30
31     bool find(int x) {
32         vx[x] = true;
33         for (int y = 1; y <= m; ++y) {
34             if (!vy[y]) {
35                 T delta = lx[x] + ly[y] - g[x][y];
36                 if (delta==0) {
37                     vy[y] = true;
38                     if (match[y] == 0 || find(match[y])) {
39                         match[y] = x;
40                         return true;
41                     }
42                 } else slack[y] = min(slack[y], delta);
43             }
44         }
45         return false;
46     }
47
48     T matching() { // maximum weight matching
49         fill(lx + 1, lx + 1 + n, numeric_limits<T>::lowest());
50         ms(ly,0);
51         ms(match,0);
52         for (int i = 1; i <= n; ++i) {
53             for (int j = 1; j <= m; ++j) lx[i] = max(lx[i], g[i][j]);
54         }
55         for (int k = 1; k <= n; ++k) {
56             fill(slack + 1, slack + 1 + m, numeric_limits<T>::max());
57             while (true) {
58                 ms(vx,0);
59                 ms(vy,0);
60                 if (find(k)) break;
61                 else {
62                     T delta = numeric_limits<T>::max();
63                     for (int i = 1; i <= m; ++i) {
64                         if (!vy[i]) delta = min(delta, slack[i]);
65                     }
66                     for (int i = 1; i <= n; ++i) {
67                         if (vx[i]) lx[i] -= delta;
68                     }
69                     for (int i = 1; i <= m; ++i) {
70                         if (vy[i]) ly[i] += delta;
71                         if (!vy[i]) slack[i] -= delta;
72                     }
73                 }
74             }
75         }
76         T result = 0;
77         for (int i = 1; i <= n; ++i) result += lx[i];
78         for (int i = 1; i <= m; ++i) result += ly[i];
79         return result;
80     }
81 };
82
83 // Ejemplo de uso anterior (comentado para pegar el algoritmo con la plantilla):
84 // void solve() {
85 //     int n, m;
86 //     cin >> n >> m;
87 //

```

```

88 //     KuhnMunkras<double> km;
89 //     km.init(n, m);
90 //
91 //     for (int i = 1; i <= n; ++i) {
92 //         for (int j = 1; j <= m; ++j) {
93 //             double c;
94 //             cin >> c;
95 //             km.add(i,j, -c);
96 //         }
97 //     }
98 //
99 //     double result = km.matching();
100 //     double total_cost = -result;
101 //     cout << total_cost << '\n';
102 // }
103 //
104 // int main() {
105 //     ios::sync_with_stdio(false);
106 //     cin.tie(nullptr);
107 //     int t = 1;
108 //     //cin >> t;
109 //     while(t--)
110 //         solve();
111 // }

```

Kosaraju Scc

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 #define fst first
6 #define snd second
7 #define ll long long
8 #define ms(a, b) memset(a, b, sizeof(a))
9
10 const int INF = 1 << 30;
11 const int MAXN = 25;
12 const int MAXM = 25;
13
14 vector<bool> visited; // keeps track of which vertices are already visited
15
16 // runs depth first search starting at vertex v.
17 // each visited vertex is appended to the output vector when dfs leaves it.
18 void dfs(int v, vector<vector<int>> &adj, vector<int> &output) {
19     visited[v] = true;
20     for (auto u : adj[v])
21         if (!visited[u])
22             dfs(u, adj, output);
23     output.push_back(v);
24 }
25
26 // input: adj -- adjacency list of G
27 // output: components -- the strongly connected components in G
28 // output: adj_cond -- adjacency list of G^SCC (by root vertices)
29 void strongly_connected_components(vector<vector<int>> &adj,
30     vector<vector<int>> &components,
31     vector<vector<int>> &adj_cond) {
32     int n = (int)adj.size();
33     components.clear();
34     adj_cond.clear();
35
36     // 1) Order the vertices by finish time in a first DFS.
37     vector<int> order;
38     visited.assign(n, false);
39     for (int i = 0; i < n; i++) {
40         if (!visited[i]) {
41             dfs(i, adj, order);
42         }
43     }
44
45     // 2) Reverse all edges to get the transpose graph.
46     vector<vector<int>> adj_rev(n);
47     for (int v = 0; v < n; v++) {
48         for (int u : adj[v]) {
49             adj_rev[u].push_back(v);
50         }
51     }
52
53     // 3) Do a second DFS in descending order of finish times (from step 1).
54     visited.assign(n, false);
55     reverse(order.begin(), order.end());
56

```

```

57 // this array will store the "root" (representative) of each SCC
58 vector<int> roots(n, -1);
59
60 for (auto v : order) {
61     if (!visited[v]) {
62         vector<int> component;
63         dfs(v, adj_rev, component);
64         // store the newly found component
65         components.push_back(component);
66         // the root is the minimum vertex in that component (arbitrary choice)
67         int root = *min_element(begin(component), end(component));
68         for (auto u : component) {
69             roots[u] = root;
70         }
71     }
72 }
73
74 // 4) Build the condensation graph (SCC DAG).
75 adj_cond.assign(n, {});
76 for (int v = 0; v < n; v++) {
77     for (auto u : adj[v]) {
78         if (roots[v] != roots[u]) {
79             adj_cond[roots[v]].push_back(roots[u]);
80         }
81     }
82 }
83 }
84
85 // Ejemplo de uso anterior (comentado para pegar el algoritmo con la plantilla):
86 // void solve() {
87 //
88 //     int n = 5, m = 5;
89 //     vector<vector<int>> adj(n);
90 //
91 //     vector<pair<int,int>> edges = {
92 //         {0,1},
93 //         {1,2},
94 //         {2,0},
95 //         {1,3},
96 //         {3,4}
97 //     };
98 //
99 //     for (auto &e : edges) {
100 //         adj[e.first].push_back(e.second);
101 //     }
102 //
103 //     vector<vector<int>> components, cond_graph;
104 //     strongly_connected_components(adj, components, cond_graph);
105 //
106 //     for (auto &comp : components) {
107 //         sort(comp.begin(), comp.end());
108 //     }
109 //
110 //     cout << "Number of SCCs: " << (int)components.size() << "\n";
111 //     for (int i = 0; i < (int)components.size(); i++) {
112 //         cout << "SCC " << i << ": ";
113 //         for (auto &v : components[i]) {
114 //             cout << v << " ";
115 //         }
116 //         cout << "\n";
117 //     }
118 //
119 //     cout << "Condensation graph adjacency list:\n";
120 //     // We'll print only non-empty adjacency to see edges
121 //     for (int i = 0; i < (int)cond_graph.size(); i++) {
122 //         if (!cond_graph[i].empty()) {
123 //             cout << "root " << i << ": ";
124 //             for (auto &v : cond_graph[i]) {
125 //                 cout << v << " ";
126 //             }
127 //             cout << "\n";
128 //         }
129 //     }
130 // }
131 //
132 // int main() {
133 //     ios::sync_with_stdio(false);
134 //     cin.tie(nullptr);
135 //     int t = 1;
136 //     //cin >> t;
137 //     while(t--)
138 //         solve();
139 // }

```

Kruskal

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 #define ll long long
5 #define endl '\n'
6
7 struct Edge {
8     ll u, v, weight;
9     bool operator<(Edge const& other) const {
10         return weight < other.weight;
11     }
12 };
13
14 struct DSU {
15     vector<int> parent, size;
16
17     DSU(int n) {
18         parent.resize(n);
19         size.assign(n, 1);
20         iota(parent.begin(), parent.end(), 0);
21     }
22
23     int find(int v) {
24         if (v == parent[v]) return v;
25         return parent[v] = find(parent[v]);
26     }
27
28     bool unite(int a, int b) {
29         a = find(a);
30         b = find(b);
31         if (a == b) return false;
32         if (size[a] < size[b]) swap(a, b);
33         parent[b] = a;
34         size[a] += size[b];
35         return true;
36     }
37 };
38
39 ll kruskal(int n, vector<Edge>& edges, vector<Edge>& result) {
40     sort(edges.begin(), edges.end());
41     DSU dsu(n);
42     ll cost = 0;
43
44     for (auto& e : edges) {
45         if (dsu.unite(e.u, e.v)) {
46             cost += e.weight;
47             result.push_back(e);
48         }
49     }
50     return cost;
51 }
52
53 // Ejemplo de uso anterior (comentado para pegar el algoritmo con la plantilla):
54 // int main() {
55 //     ios::sync_with_stdio(false);
56 //     cin.tie(nullptr);
57 //
58 //     int n, m;
59 //     cin >> n >> m;
60 //
61 //     vector<Edge> edges;
62 //     edges.reserve(m);
63 //
64 //     for (int i = 0; i < m; i++) {
65 //         ll u, v, w;
66 //         cin >> u >> v >> w;
67 //         u--, v--;
68 //         edges.push_back({u, v, w});
69 //     }
70 //
71 //     vector<Edge> result;
72 //     ll total_cost = kruskal(n, edges, result);
73 //
74 //     if(result.size() == n-1) {
75 //         cout << total_cost << endl;
76 //     } else {
77 //         cout << "IMPOSSIBLE" << endl;
78 //     }
79 //
80 //     // Optional: print MST edges
81 //     for (auto &e : result)
```

```

82 // //      cout << e.u + 1 << " " << e.v + 1 << " " << e.weight << endl;
83 // }

```

Topological Sort

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 #define fst first
6 #define snd second
7 #define ll long long
8
9 const int INF = 1 << 30;
10 const int MAXN = 1e5+5;
11
12 vector<int> adj[MAXN];
13 vector<int> topo;
14
15 int n,m;
16
17 void dfs(int u, vector<int> &visited) {
18     visited[u] = 1;
19     for (int &v : adj[u]) {
20         if (!visited[v])
21             dfs(v, visited);
22         else if(visited[v] == 1){
23             cout << "IMPOSSIBLE\n";
24             exit(0);
25         }
26     }
27     visited[u] = 2;
28     topo.push_back(u);
29 }
30
31 void topological_sort() {
32     topo.clear();
33     vector<int> visited(n+1,0);
34
35     for (int i = 1; i <= n; i++)
36         if (!visited[i])
37             dfs(i, visited);
38
39     reverse(topo.begin(), topo.end());
40 }
41
42 // Ejemplo de uso anterior (comentado para pegar el algoritmo con la plantilla):
43 // void solve() {
44 //     cin >> n >> m;
45 //     for(int i = 0; i <= n; i++) adj[i].clear();
46 //     while(m--) {
47 //         int a,b;
48 //         cin >> a >> b;
49 //         adj[a].push_back(b);
50 //     }
51 //     topological_sort();
52 //     for (int x : topo) cout << x << " ";
53 //     cout << "\n";
54 // }
55 //
56 // int main() {
57 //     int t = 1;
58 //     // cin >> t;
59 //     while(t--)
60 //         solve();
61 // }

```

Math

Binary Exponentiation

```

1 #include <bits/stdc++.h>
2
3 #define ll long long
4
5 ll binpow(ll a, ll b) {
6     if (b == 0)
7         return 1;
8     ll res = binpow(a, b / 2);
9     if (b % 2)
10         return res * res * a;

```

```

11     else
12         return res * res;
13 }
14
15 ll binpowIt(ll a, ll b) {
16     ll res = 1;
17     while (b > 0) {
18         if (b & 1)
19             res = res * a;
20         a = a * a;
21         b >>= 1;
22     }
23     return res;
24 }
25
26 int power(ll x, ll y, ll M)
27 {
28     if (y == 0)
29         return 1;
30
31     int p = power(x, y / 2, M) % M;
32     p = (p * p) % M;
33
34     return (y % 2 == 0) ? p : (x * p) % M;
35 }
36 // inverse modular power(A, M - 2, M)

```

Expected Value

```

1 // Expected Value & Probability (Esperanza Matematica)
2 // 1. Linealidad de la Esperanza:  $E[X + Y] = E[X] + E[Y]$  (siempre valido, aun dependientes)
3 // 2. Variable indicadora:  $E[I_A] = P(A)$ 
4 // 3. Ensayos geometricos (primer exito con prob p):  $E = 1/p$ 
5 // 4. Coupon Collector (recolectar n cupones):  $E = n * \sum_{i=1}^n (1/i) = n * H_n$ 
6
7 // --- Sistema Lineal para Esperanza en Grafos de Estados / Markov -----
8 //  $E[u] = 1 + \sum_{\{v\}} P(u \rightarrow v) * E[v]$ 
9 // Se transforma en:  $E[u] - \sum_{\{v\}} P(u \rightarrow v) * E[v] = 1$  (con  $E[\text{target}] = 0$ )
10 // Resuelto con Eliminacion Gaussiana  $O(N^3)$ 
11
12 const ld EPS = 1e-11;
13
14 // Retorna 1 si tiene solucion unica, 0 si no o infinitas. Solucion en ans.
15 int gauss(vector<vector<ld>> a, vector<ld>& ans) {
16     int n = sz(a), m = sz(a[0]) - 1;
17     vi where(m, -1);
18     for (int col = 0, row = 0; col < m && row < n; ++col) {
19         int sel = row;
20         for (int i = row; i < n; ++i)
21             if (fabs1(a[i][col]) > fabs1(a[sel][col])) sel = i;
22         if (fabs1(a[sel][col]) < EPS) continue;
23         for (int i = col; i <= m; ++i) swap(a[sel][i], a[row][i]);
24         where[col] = row;
25
26         for (int i = 0; i < n; ++i) {
27             if (i != row) {
28                 ld c = a[i][col] / a[row][col];
29                 for (int j = col; j <= m; ++j) a[i][j] -= a[row][j] * c;
30             }
31         }
32         ++row;
33     }
34     ans.assign(m, 0);
35     rep(i, m) {
36         if (where[i] != -1) ans[i] = a[where[i]][m] / a[where[i]][i];
37     }
38     rep(i, n) {
39         ld sum = 0;
40         rep(j, m) sum += ans[j] * a[i][j];
41         if (fabs1(sum - a[i][m]) > EPS) return 0;
42     }
43     return 1;
44 }
45
46 // Construccion de matriz para n estados (0..n-1), absorbiendo en target
47 // adj[u] = lista de {v, prob}
48 vector<ld> expectedSteps(int n, int target, const vector<vector<pair<int, ld>>& adj) {
49     vector<vector<ld>> A(n, vector<ld>(n + 1, 0));
50     rep(u, n) {
51         if (u == target) {
52             A[u][u] = 1;
53             A[u][n] = 0; //  $E[\text{target}] = 0$ 
54             continue;
55         }
56     }
57 }

```

```

55     }
56     A[u][u] = 1.0;
57     A[u][n] = 1.0; // sumatoria de costo/paso base = 1
58     for (auto& edge : adj[u]) {
59         int v = edge.ff;
60         ld p = edge.ss;
61         A[u][v] -= p;
62     }
63 }
64 vector<ld> E;
65 gauss(A, E);
66 return E;
67 }

```

FFT

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  #define ll long long
5  const double PI = acos(-1);
6
7  namespace FFT {
8      static vector<int> rev;
9      static vector<complex<double>> raices{{0,0},{1,0}};
10
11      void fft(vector<complex<double>>& a, bool invertido) {
12          int n = a.size();
13
14          if (rev.size() != n) {
15              int k = __builtin_ctz(n); // log2(n)
16              rev.assign(n, 0);
17              for (int i = 0; i < n; i++) {
18                  rev[i] = (rev[i >> 1] >> 1) | ((i & 1) << (k - 1));
19              }
20          }
21
22          for (int i = 0; i < n; i++) {
23              if (i < rev[i]) swap(a[i], a[rev[i]]);
24          }
25
26          if (raices.size() < n) {
27              int k = __builtin_ctz(raices.size());
28              raices.resize(n);
29              while ((1 << k) < n) {
30                  double angulo = 2 * PI / (1 << (k + 1));
31                  for (int i = 1 << (k - 1); i < (1 << k); i++) {
32                      raices[2 * i] = raices[i];
33                      double ang = angulo * (2 * i + 1 - (1 << k));
34                      raices[2 * i + 1] = complex<double>(cos(ang), sin(ang));
35                  }
36                  k++;
37              }
38          }
39
40          for (int len = 1; len < n; len = len << 1) {
41              for (int i = 0; i < n; i += 2 * len) {
42                  for (int j = 0; j < len; j++) {
43                      complex<double> u = a[i + j];
44                      complex<double> v = a[i + j + len] * raices[len + j];
45                      a[i + j] = u + v;
46                      a[i + j + len] = u - v;
47                  }
48              }
49          }
50
51          if (invertido) {
52              reverse(a.begin() + 1, a.end());
53              for (auto &x : a) x /= n;
54          }
55      }
56  } // namespace FFT
57
58  vector<ll> multiplicacion_polinomio(const vector<ll>& A, const vector<ll>& B) {
59      int n1 = A.size(), n2 = B.size();
60      if (n1 == 0 || n2 == 0) return vector<ll>();
61
62      int n = 1;
63      while (n < n1 + n2 - 1) n = n << 1;
64      vector<complex<double>> fa(n), fb(n);
65      for (int i = 0; i < n1; i++) fa[i] = complex<double>(A[i], 0);
66      for (int i = 0; i < n2; i++) fb[i] = complex<double>(B[i], 0);
67

```

```

68     FFT::fft(fa, false);
69     FFT::fft(fb, false);
70     for (int i = 0; i < n; i++) fa[i] *= fb[i];
71     FFT::fft(fa, true);
72
73     vector<ll> res(n1 + n2 - 1);
74     for (int i = 0; i < res.size(); i++) {
75         res[i] = llround(fa[i].real()); // Redondeo
76     }
77     return res;
78 }
79
80 // Ejemplo de uso anterior (comentado para pegar el algoritmo con la plantilla):
81 // int main() {
82 //     vector<ll> A = {1, 2, 3}; // 1 + 2x + 3x^2
83 //     vector<ll> B = {2, 0, 1}; // 2 + 0x + 1x^2
84 //     vector<ll> C = multiplicacion_polinomio(A, B);
85 //     return 0;
86 // }

```

Matrix Exponentiation

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 #define ll long long
6 const int MOD = 1e9+7;
7
8 struct Matrix {
9     int n;
10    vector<vector<ll>> mat;
11
12    Matrix(int size) : n(size), mat(size, vector<ll>(size)) {}
13
14    // Identity matrix
15    static Matrix identity(int size) {
16        Matrix I(size);
17        for (int i = 0; i < size; i++)
18            I.mat[i][i] = 1;
19        return I;
20    }
21
22    // Matrix multiplication
23    Matrix operator*(const Matrix& other) const {
24        Matrix product(n);
25        for (int i = 0; i < n; ++i)
26            for (int k = 0; k < n; ++k)
27                for (int j = 0; j < n; ++j) {
28                    product.mat[i][j] += mat[i][k] * other.mat[k][j];
29                    product.mat[i][j] %= MOD;
30                }
31        return product;
32    }
33
34    // Matrix exponentiation
35    Matrix pow(ll power) const {
36        Matrix result = identity(n);
37        Matrix base = *this;
38
39        while (power > 0) {
40            if (power & 1)
41                result = result * base;
42            base = base * base;
43            power >>= 1;
44        }
45        return result;
46    }
47 };
48
49 ll fibonacci(int n) {
50     if (n == 0) return 0;
51     Matrix base(2);
52     base.mat = {{1, 1}, {1, 0}};
53     base = base.pow(n - 1);
54     return base.mat[0][0]; // F(n)
55 }
56
57 // Ejemplo de uso anterior (comentado para pegar el algoritmo con la plantilla):
58 // int main() {
59 //     for(int i = 0; i < 25; i++)
60 //         cout << fibonacci(i) << " ";
61 // }

```

NTT

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 #define ll long long
5
6 namespace NTT {
7     // MOD debe ser un primo de la forma c * 2^k + 1
8     // 998244353 = 119 * 2^23 + 1. Raíz primitiva G = 3.
9     const ll MOD = 998244353;
10    const ll G = 3; // Raíz primitiva para MOD
11
12    ll fast_pow(ll a, ll b) {
13        ll res = 1;
14        a %= MOD;
15        while (b > 0) {
16            if (b & 1) res = (res * a) % MOD;
17            a = (a * a) % MOD;
18            b >>= 1;
19        }
20        return res;
21    }
22
23    ll inv(ll a) {
24        return fast_pow(a, MOD - 2);
25    }
26
27    static vector<int> rev;
28    static vector<ll> w;
29
30    void precompute(int n) {
31        if (rev.size() == n) return;
32        int k = __builtin_ctz(n);
33        rev.assign(n, 0);
34        for (int i = 0; i < n; i++) {
35            rev[i] = (rev[i >> 1] >> 1) | ((i & 1) << (k - 1));
36        }
37
38        w.resize(n);
39        w[0] = 1;
40        ll g_n = fast_pow(G, (MOD - 1) / n);
41        for (int i = 1; i < n; i++) {
42            w[i] = (w[i - 1] * g_n) % MOD;
43        }
44    }
45
46    void fft(vector<ll>& a, bool invertido) {
47        int n = a.size();
48        precompute(n);
49
50        for (int i = 0; i < n; i++) {
51            if (i < rev[i]) swap(a[i], a[rev[i]]);
52        }
53
54        for (int len = 1; len < n; len <= 1) {
55            int step = n / (2 * len);
56            for (int i = 0; i < n; i += 2 * len) {
57                for (int j = 0; j < len; j++) {
58                    ll u = a[i + j];
59                    ll v = (a[i + j + len] * w[j * step]) % MOD;
60
61                    a[i + j] = (u + v) % MOD;
62                    a[i + j + len] = (u - v + MOD) % MOD;
63                }
64            }
65        }
66
67        if (invertido) {
68            reverse(a.begin() + 1, a.end());
69            ll n_inv = inv(n);
70            for (auto &x : a) {
71                x = (x * n_inv) % MOD;
72            }
73        }
74    }
75 } // namespace NTT
76
77 vector<ll> multiplicacion_polinomio_mod(const vector<ll>& A, const vector<ll>& B) {
78     int n1 = A.size(), n2 = B.size();
79     if (n1 == 0 || n2 == 0) return vector<ll>();
80
81     int n = 1;
```

```

82     while (n < n1 + n2 - 1) n = n << 1;
83
84     vector<ll> fa(n), fb(n);
85     for (int i = 0; i < n1; i++) fa[i] = (A[i] % NTT::MOD + NTT::MOD) % NTT::MOD;
86     for (int i = 0; i < n2; i++) fb[i] = (B[i] % NTT::MOD + NTT::MOD) % NTT::MOD;
87
88     NTT::fft(fa, false);
89     NTT::fft(fb, false);
90
91     for (int i = 0; i < n; i++) {
92         fa[i] = (fa[i] * fb[i]) % NTT::MOD;
93     }
94
95     NTT::fft(fa, true);
96
97     fa.resize(n1 + n2 - 1);
98     return fa;
99 }
100
101 // Ejemplo de uso anterior (comentado para pegar el algoritmo con la plantilla):
102 // int main() {
103 //     vector<ll> A = {1, 2, 3}; // 1 + 2x + 3x^2
104 //     vector<ll> B = {2, 0, 1}; // 2 + 0x + 1x^2
105 //     vector<ll> C = multiplicacion_polinomio_mod(A, B);
106 //     return 0;
107 // }

```

Strings

Aho Corasick

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 // ALPHA is a compile-time constant: one contiguous allocation for all nodes.
5 // toIdx maps a character to [0, ALPHA) — swap freely for any alphabet.
6 template<int ALPHA>
7 struct AhoCorasick {
8     function<int(char)> toIdx;
9     vector<array<int, ALPHA>> go;
10    vector<int> fail, dict, output;
11    // output[v] = pattern index ending at v (-1 if none)
12
13    AhoCorasick(function<int(char)> toIdx) : toIdx(toIdx) {
14        go.emplace_back(); go.back().fill(-1);
15        fail.push_back(0);
16        dict.push_back(-1);
17        output.push_back(-1);
18    }
19
20    int insert(const string& s, int patId) {
21        int cur = 0;
22        for (char c : s) {
23            int ch = toIdx(c);
24            if (go[cur][ch] == -1) {
25                go[cur][ch] = go.size();
26                go.emplace_back(); go.back().fill(-1);
27                fail.push_back(0);
28                dict.push_back(-1);
29                output.push_back(-1);
30            }
31            cur = go[cur][ch];
32        }
33        output[cur] = patId;
34        return cur;
35    }
36
37    // Build failure and dictionary links (call after all insertions)
38    void build() {
39        queue<int> q;
40        for (int c = 0; c < ALPHA; c++) {
41            if (go[0][c] == -1) {
42                go[0][c] = 0; // missing char at root loops back to root
43            } else {
44                fail[go[0][c]] = 0;
45                q.push(go[0][c]);
46            }
47        }
48        while (!q.empty()) {
49            int u = q.front(); q.pop();
50            dict[u] = (output[fail[u]] != -1) ? fail[u] : dict[fail[u]];

```

```

51         for (int c = 0; c < ALPHA; c++) {
52             if (go[u][c] == -1) {
53                 go[u][c] = go[fail[u]][c]; // compressed goto
54             } else {
55                 fail[go[u][c]] = go[fail[u]][c];
56                 q.push(go[u][c]);
57             }
58         }
59     }
60 }
61
62 void search(const string& text, const vector<string>& pats,
63            function<void(int,int)> cb) {
64     int cur = 0;
65     for (int i = 0; i < (int)text.size(); i++) {
66         cur = go[cur][toIdx(text[i])];
67         for (int v = cur; v > 0; v = dict[v])
68             if (output[v] >= 0)
69                 cb(i - (int)pats[output[v]].size() + 1, output[v]);
70     }
71 }
72
73 };
74
75 // Ejemplo de uso anterior (main comentado para compilar con la plantilla):
76 // int main() {
77 //     vector<string> patterns = {"he", "she", "his", "hers"};
78 //     AhoCorasick<26> ac([](char c) { return c - 'a'; });
79 //     for (int i = 0; i < (int)patterns.size(); ++i) ac.insert(patterns[i], i);
80 //     ac.build();
81 //     ac.search("ahishers", patterns, [&](int pos, int id) {
82 //         cout << patterns[id] << " at " << pos << "\n";
83 //     });
84 // }

```

Bit Trie

```

1 struct BitTrie {
2     static const int B = 30; // bits (use 62 with ll values)
3     struct Node {
4         int nxt[2] = {-1, -1}, cnt = 0;
5     };
6     vector<Node> t;
7     BitTrie() : t(1) {}
8     void insert(int v, int d = 1) { // d = 1 insert, d = -1 erase (the value must exist)
9         int x = 0;
10        for (int b = B-1; b >= 0; b--) {
11            int k = v>>b & 1;
12            if (t[x].nxt[k] < 0) { t[x].nxt[k] = t.size(); t.emplace_back(); }
13            x = t[x].nxt[k];
14            t[x].cnt += d;
15        }
16    }
17    int maxXor(int v) { // max of v ^ u over u in the set (set must be non-empty)
18        int x = 0, res = 0;
19        for (int b = B-1; b >= 0; b--) {
20            int k = v>>b & 1;
21            int y = t[x].nxt[k^1];
22            if (y >= 0 && t[y].cnt > 0) { res |= 1<<b; x = y; }
23            else x = t[x].nxt[k];
24        }
25        return res;
26    }
27    int countLess(int v, int lim) { // number of u in the set with (v ^ u) < lim
28        int x = 0, res = 0;
29        for (int b = B-1; b >= 0 && x >= 0; b--) {
30            int k = v>>b & 1, l = lim>>b & 1;
31            if (l) {
32                int y = t[x].nxt[k];
33                if (y >= 0) res += t[y].cnt; // xor bit 0 < 1, whole subtree counts
34                x = t[x].nxt[k^1];
35            } else x = t[x].nxt[k];
36        }
37        return res;
38    }
39 };

```

Double Hash

```

1 #include <bits/stdc++.h>
2 using namespace std;
3

```

```

4 // Double hashing: use two independent hash functions
5 // to make collision probability negligible ( $\sim 1/P_1 * 1/P_2$ )
6
7 struct DoubleHash {
8     static const long long MOD1 = 1e9 + 7;
9     static const long long MOD2 = 1e9 + 9;
10    static const long long B1 = 131;
11    static const long long B2 = 137;
12
13    vector<long long> h1, h2, pw1, pw2;
14    int n;
15
16    DoubleHash(const string& s) : n(s.size()),
17        h1(s.size()+1,0), h2(s.size()+1,0),
18        pw1(s.size()+1,1), pw2(s.size()+1,1)
19    {
20        for (int i = 0; i < n; i++) {
21            h1[i+1] = (h1[i] * B1 + s[i]) % MOD1;
22            h2[i+1] = (h2[i] * B2 + s[i]) % MOD2;
23            pw1[i+1] = pw1[i] * B1 % MOD1;
24            pw2[i+1] = pw2[i] * B2 % MOD2;
25        }
26    }
27
28    // Returns a pair (hash1, hash2) for s[l..r]
29    pair<long long, long long> query(int l, int r) const {
30        long long q1 = (h1[r+1] - h1[l] * pw1[r-l+1] % MOD1 + MOD1) % MOD1;
31        long long q2 = (h2[r+1] - h2[l] * pw2[r-l+1] % MOD2 + MOD2) % MOD2;
32        return {q1, q2};
33    }
34
35    bool equal(int l1, int r1, int l2, int r2) const {
36        return query(l1, r1) == query(l2, r2);
37    }
38 };
39
40 // Ejemplo de uso anterior (main comentado para compilar con la plantilla):
41 // int main() {
42 //     string s = "abracadabra";
43 //     DoubleHash h(s);
44 //     cout << h.query(0, 2).first << " " << h.query(0, 2).second << "\n";
45 //     cout << boolalpha << h.equal(0, 2, 7, 9) << "\n";
46 // }

```

Hash

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 // Single polynomial hash of a string
5 // hash(s) = s[0]*B^(n-1) + s[1]*B^(n-2) + ... + s[n-1]*B^0 (mod P)
6 // Equivalent iterative: hash = hash*B + s[i]
7
8 const long long MOD = 1e9 + 7;
9 const long long BASE = 131;
10
11 long long hash_str(const string& s) {
12     long long h = 0;
13     for (char c : s) {
14         h = (h * BASE + c) % MOD;
15     }
16     return h;
17 }
18
19 // Precompute prefix hashes and powers for O(1) substring queries
20 struct HashTable {
21     vector<long long> h, pw;
22     long long mod, base;
23
24     HashTable(const string& s, long long B = BASE, long long M = MOD)
25         : h(s.size() + 1, 0), pw(s.size() + 1, 1), mod(M), base(B)
26     {
27         for (int i = 0; i < (int)s.size(); i++) {
28             h[i + 1] = (h[i] * base + s[i]) % mod;
29             pw[i + 1] = pw[i] * base % mod;
30         }
31     }
32
33     // Hash of s[l..r] (0-indexed, inclusive)
34     long long query(int l, int r) const {
35         return (h[r + 1] - h[l] * pw[r - l + 1] % mod + mod) % mod;
36     }
37 };

```

```

38
39 // Ejemplo de uso anterior (main comentado para compilar con la plantilla):
40 // int main() {
41 //     string s = "competitive";
42 //     HashTable h(s);
43 //     cout << hash_str(s) << "\n";
44 //     cout << h.query(0, 3) << "\n"; // hash of s[0..3]
45 // }

```

KMP Automaton

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 #define ll long long
6 #define endl '\n'
7
8 vector<int> prefix_function(string s) {
9     int n = (int)s.length();
10    vector<int> pi(n);
11    for (int i = 1; i < n; i++) {
12        int j = pi[i-1];
13        while (j > 0 && s[i] != s[j])
14            j = pi[j-1];
15        if (s[i] == s[j])
16            j++;
17        pi[i] = j;
18    }
19    return pi;
20 }
21
22 void compute_automaton(string s, vector<vector<int>>& aut) {
23     s += '#';
24     int n = s.size();
25     vector<int> pi = prefix_function(s);
26     aut.assign(n, vector<int>(26));
27     for (int i = 0; i < n; i++) {
28         for (int c = 0; c < 26; c++) {
29             if (i > 0 && 'a' + c != s[i])
30                 aut[i][c] = aut[pi[i-1]][c];
31             else
32                 aut[i][c] = i + ('a' + c == s[i]);
33         }
34     }
35 }
36
37 const int MAXN = 1e4+5;
38 const int MAXK = 1e3+5;
39 const int MOD = 1e9+7;
40
41
42 int n,k;
43 string pattern;
44 vector<vector<int>> aut;
45
46 long long binpow(long long a, long long b)
47 {
48     long long result = 1;
49     while (b) {
50         if (b & 1)
51             result = (result * a) % MOD;
52         a = (a * a) % MOD;
53         b >>= 1;
54     }
55     return result;
56 }
57
58 ll dp[MAXN][MAXK];
59 ll fdp(int pos, int state) {
60     bool found = state == pattern.size();
61     if(pos == n) {
62         return found;
63     }
64
65     ll &val = dp[pos][state];
66     if(val != -1) return val;
67
68     val = 0;
69
70     if(found) {
71         ll l = n-pos;
72         return val = binpow(k, l);
73     }

```

```

73 }
74
75 for(int i = 0; i < k; i++) {
76     val = (val + fdp(pos+1,aut[state][i])) % MOD;
77 }
78
79 return val;
80 }
81
82 // Ejemplo de uso anterior (comentado para pegar el algoritmo con la plantilla):
83 // void solve() {
84 //     memset(dp, -1, sizeof dp);
85 //     cin >> n >> k >> pattern;
86 //
87 //     compute_automaton(pattern, aut);
88 //
89 //     cout << fdp(0,0) << endl;
90 // }
91 //
92 // int main() {
93 //     solve();
94 // }

```

KMP

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 vector<int> buildFail(const string& P) {
5     int m = P.size();
6     vector<int> fail(m, 0);
7     for (int i = 1, j = 0; i < m; i++) {
8         while (j > 0 && P[i] != P[j]) j = fail[j - 1];
9         if (P[i] == P[j]) j++;
10        fail[i] = j;
11    }
12    return fail;
13 }
14
15 // -- Application 1: Count non-overlapping occurrences -----
16 int countNonOverlapping(const string& T, const string& P) {
17     auto fail = buildFail(P);
18     int count = 0, j = 0, m = P.size();
19     for (char c : T) {
20         while (j > 0 && c != P[j]) j = fail[j - 1];
21         if (c == P[j]) j++;
22         if (j == m) { count++; j = 0; } // reset instead of fail[m-1]
23     }
24     return count;
25 }
26
27 // -- Application 2: Shortest period of a string -----
28 // The period of S is the smallest p such that S[i] == S[i % p] for all i.
29 // Using KMP: period = m - fail[m-1] (if m % (m-fail[m-1]) == 0, else period = m)
30 int period(const string& S) {
31     auto fail = buildFail(S);
32     int m = S.size();
33     int p = m - fail[m - 1];
34     return (m % p == 0) ? p : m;
35 }
36
37 // -- Application 3: Concatenation search (good for multiple patterns) -----
38 // Search for P in T by building the "combined" string P + '#' + T,
39 // then checking where fail[i] == m in the combined string.
40 vector<int> kmpConcat(const string& T, const string& P) {
41     string S = P + '#' + T; // '#' is a separator not in the alphabet
42     auto fail = buildFail(S);
43     int m = P.size();
44     vector<int> matches;
45     for (int i = m + 1; i < (int)S.size(); i++)
46         if (fail[i] == m)
47             matches.push_back(i - 2 * m); // convert back to T-index
48     return matches;
49 }
50
51 // Ejemplo de uso anterior (main comentado para compilar con la plantilla):
52 // int main() {
53 //     string text = "ababa", pattern = "aba";
54 //     cout << countNonOverlapping(text, pattern) << "\n";
55 //     cout << period(text) << "\n";
56 //     for (int pos : kmpConcat(text, pattern)) cout << pos << " ";
57 //     cout << "\n";
58 // }

```

Rolling Hash

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const long long MOD = 1e9 + 7;
5 const long long BASE = 131;
6
7 long long power(long long base, long long exp, long long mod) {
8     long long result = 1;
9     base %= mod;
10    while (exp > 0) {
11        if (exp & 1) result = result * base % mod;
12        base = base * base % mod;
13        exp >>= 1;
14    }
15    return result;
16 }
17
18 // Compute hash of every substring of length k in O(n) total
19 vector<long long> rolling_hash(const string& s, int k) {
20     int n = s.size();
21     if (n < k) return {};
22
23     vector<long long> result;
24     long long h = 0;
25     long long base_k = power(BASE, k - 1, MOD); // B^(k-1)
26
27     // Initial window [0, k-1]
28     for (int i = 0; i < k; i++)
29         h = (h * BASE + s[i]) % MOD;
30     result.push_back(h);
31
32     for (int i = 1; i + k - 1 < n; i++) {
33         // Remove s[i-1], add s[i+k-1]
34         h = (h - (long long)s[i - 1] * base_k % MOD + MOD) % MOD;
35         h = (h * BASE + s[i + k - 1]) % MOD;
36         result.push_back(h);
37     }
38
39     return result;
40 }
41
42 // Find all occurrences of pattern p in text t using rolling hash
43 vector<int> rabin_karp(const string& t, const string& p) {
44     int n = t.size(), m = p.size();
45     long long ph = 0;
46     for (char c : p) ph = (ph * BASE + c) % MOD;
47
48     vector<long long> th = rolling_hash(t, m);
49     vector<int> matches;
50     for (int i = 0; i < (int)th.size(); i++)
51         if (th[i] == ph)
52             if (t.substr(i, m) == p) // confirm match (avoid hash collision)
53                 matches.push_back(i);
54     return matches;
55 }
56
57 // Ejemplo de uso anterior (main comentado para compilar con la plantilla):
58 // int main() {
59 //     string text = "abacaba", pattern = "aba";
60 //     for (int pos : rabin_karp(text, pattern)) cout << pos << " ";
61 //     cout << "\n";
62 // }
```

String Hashing

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 #define ll long long
6
7 const int MAXN = 1e5+5;
8 const int BASE = 31;
9 const int M = 1e9 + 7;
10
11 int pot[MAXN];
12
13 void compute_powers(int n) {
14     int p_pow = 1;
15     for (int i = 0; i < n; i++) {
```

```

16     pot[i] = p_pow;
17     p_pow = (p_pow * BASE) % M;
18 }
19 }
20
21 vector<ll> compute_hash(string const& s) {
22     int n = s.size();
23     vector<ll> hash_values(n + 5);
24     hash_values[0] = 0;
25     for (int i = 1; i <= n; i++) {
26         hash_values[i] = (hash_values[i - 1] * BASE + s[i - 1]) % M;
27     }
28     return hash_values;
29 }
30
31 ll get_hash(vector<ll> &hash_values, int a, int b) {
32     return (hash_values[b] - (hash_values[a - 1] * pot[b - a + 1]) % M + M) % M;
33 }
34
35 // Ejemplo de uso anterior (comentado para pegar el algoritmo con la plantilla):
36 // int main() {
37 //     string word = "ALLEY";
38 //     compute_powers(word.size());
39 //     vector<ll> hash = compute_hash(word);
40 //
41 //     for (int i = 1; i <= word.size(); i++) {
42 //         cout << hash[i] << ' ';
43 //     }
44 //     cout << endl;
45 //     for (int i = 1; i <= word.size(); i++) {
46 //         cout << word[i - 1] << ": " << get_hash(hash, i, i) << endl;
47 //     }
48 //
49 //     return 0;
50 // }

```

Suffix Array

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 vector<int> suffix_array(const string& s) {
5     int n = s.size();
6     vector<int> sa(n), rank_(n), tmp(n);
7     iota(sa.begin(), sa.end(), 0);
8     for (int i = 0; i < n; i++) rank_[i] = s[i];
9     for (int k = 1; k < n; k <= 1) {
10         auto key = [&](int i) -> pair<int, int> {
11             return {rank_[i], i + k < n ? rank_[i + k] : -1};
12         };
13         sort(sa.begin(), sa.end(), [&](int a, int b){ return key(a) < key(b); });
14         tmp[sa[0]] = 0;
15         for (int i = 1; i < n; i++)
16             tmp[sa[i]] = tmp[sa[i-1]] + (key(sa[i]) != key(sa[i-1]));
17         rank_ = tmp;
18         if (rank_[sa[n-1]] == n-1) break;
19     }
20     return sa;
21 }
22
23 vector<int> kasai_lcp(const string& s, const vector<int>& sa) {
24     int n = sa.size();
25     vector<int> rank_(n), lcp(n, 0);
26     for (int i = 0; i < n; i++) rank_[sa[i]] = i;
27     int h = 0;
28     for (int i = 0; i < n; i++) {
29         if (rank_[i] > 0) {
30             int j = sa[rank_[i] - 1];
31             while (i + h < n && j + h < n && s[i+h] == s[j+h]) h++;
32             lcp[rank_[i]] = h;
33             if (h > 0) h--;
34         }
35     }
36     return lcp;
37 }
38
39 // -- 1. Pattern search: O(M log N) -----
40 // Binary search on SA: all matches are a contiguous range [lo, hi).
41 pair<int, int> search_range(const string& s, const vector<int>& sa, const string& P) {
42     int n = sa.size(), m = P.size();
43     int lo = (int)(lower_bound(sa.begin(), sa.end(), 0, [&](int a, int) {
44         return s.compare(a, m, P) < 0;
45     }) - sa.begin());

```

```

46     int hi = (int)(upper_bound(sa.begin(), sa.end(), 0, [&](int, int a) {
47         return s.compare(a, m, P) > 0;
48     }) - sa.begin());
49     return {lo, hi};
50 }
51
52 // -- 2. Count distinct substrings -----
53 // Total substrings = N*(N+1)/2. Subtract duplicates counted by LCP array.
54 long long distinct_substrings(const string& s) {
55     auto sa = suffix_array(s);
56     auto lcp = kasai_lcp(s, sa);
57     int n = s.size();
58     long long total = (long long)n * (n + 1) / 2;
59     for (int v : lcp) total -= v;
60     return total;
61 }
62
63 // -- 3. Longest repeated substring -----
64 // The maximum value in the LCP array.
65 string longest_repeated(const string& s) {
66     auto sa = suffix_array(s);
67     auto lcp = kasai_lcp(s, sa);
68     int best = *max_element(lcp.begin(), lcp.end());
69     int idx = max_element(lcp.begin(), lcp.end()) - lcp.begin();
70     return best > 0 ? s.substr(sa[idx], best) : "";
71 }
72
73 // Ejemplo de uso anterior (main comentado para compilar con la plantilla):
74 // int main() {
75 //     string s = "banana";
76 //     vector<int> sa = suffix_array(s);
77 //     for (int pos : sa) cout << pos << " ";
78 //     cout << "\ndistinct substrings = " << distinct_substrings(s) << "\n";
79 //     cout << "longest repeated = " << longest_repeated(s) << "\n";
80 // }

```

Suffix Tree

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 struct SuffixTree {
4     struct Node {
5         int ch[26], link, start, end, cnt;
6         // end = INF for leaves (extends with each new char)
7         Node(int s, int e) : link(0), start(s), end(e), cnt(0) {
8             fill(ch, ch + 26, -1);
9         }
10        int len(int phase) { return min(end, phase + 1) - start; }
11    };
12
13    string s;
14    vector<Node> t;
15    int last, n;
16    int activeNode, activeEdge, activeLen, remaining;
17    int need;
18
19    SuffixTree() {}
20
21    void setSuffixLink(int v) {
22        if (need != -1) t[need].link = v;
23        need = v;
24    }
25
26    bool walkDown(int v) {
27        if (activeLen >= t[v].len(n - 1)) {
28            activeEdge += t[v].len(n - 1);
29            activeLen -= t[v].len(n - 1);
30            activeNode = v;
31            return true;
32        }
33        return false;
34    }
35
36    void extend(int pos) {
37        t[0].end = pos + 1; // implicit: all leaves extend
38        remaining++;
39        need = -1;
40
41        while (remaining > 0) {
42            if (activeLen == 0) activeEdge = pos;
43            int ae = s[activeEdge] - 'a';
44
45            if (t[activeNode].ch[ae] == -1) {

```

```

46         // Rule 2: create new leaf
47         t[activeNode].ch[ae] = t.size();
48         t.emplace_back(pos, INT_MAX);
49         setSuffixLink(activeNode);
50     } else {
51         int nxt = t[activeNode].ch[ae];
52         if (walkDown(nxt)) continue;
53
54         if (s[t[nxt].start + activeLen] == s[pos]) {
55             // Rule 3: already exists — just extend active len
56             activeLen++;
57             setSuffixLink(activeNode);
58             break;
59         }
60
61         // Rule 2: split
62         int split = t.size();
63         t.emplace_back(t[nxt].start, t[nxt].start + activeLen);
64         t[activeNode].ch[ae] = split;
65
66         t[split].ch[s[pos] - 'a'] = t.size();
67         t.emplace_back(pos, INT_MAX);
68         t[nxt].start += activeLen;
69         t[split].ch[s[t[nxt].start] - 'a'] = nxt;
70         setSuffixLink(split);
71     }
72
73     remaining--;
74     if (activeNode == 0 && activeLen > 0) {
75         activeLen--;
76         activeEdge = pos - remaining + 1;
77     } else {
78         activeNode = t[activeNode].link ? t[activeNode].link : 0;
79     }
80 }
81 n = pos + 1;
82 }
83
84 void build(const string& str) {
85     s = str + "$";
86     n = 0;
87     t.clear();
88     t.emplace_back(0, 0); // root
89     last = activeNode = activeLen = activeEdge = remaining = 0;
90     need = -1;
91     for (int i = 0; i < (int)s.size(); i++) extend(i);
92 }
93
94 // DFS to collect suffix indices at leaves
95 void dfs(int v, int depth, vector<int>& res) {
96     bool leaf = true;
97     for (int c = 0; c < 26; c++) {
98         if (t[v].ch[c] == -1) continue;
99         leaf = false;
100         dfs(t[v].ch[c], depth + t[t[v].ch[c]].len(n - 1), res);
101     }
102     if (leaf) {
103         // leaf: suffix index = n - 1 - depth (n includes '$')
104         res.push_back(n - 1 - depth);
105     }
106 }
107
108 // Find all occurrences of pattern P in O(|P| + occ)
109 vector<int> search(const string& p) {
110     int v = 0, i = 0;
111     while (i < (int)p.size()) {
112         int c = p[i] - 'a';
113         if (t[v].ch[c] == -1) return {};
114         int nxt = t[v].ch[c];
115         for (int j = t[nxt].start; j < t[nxt].end && i < (int)p.size(); j++, i++)
116             if (s[j] != p[i]) return {};
117         v = nxt;
118     }
119     vector<int> res;
120     dfs(v, 0, res);
121     sort(res.begin(), res.end());
122     return res;
123 }
124 };
125
126 // --- Longest Repeated Substring ---
127 // DFS: track string depth (total chars from root).
128 // Deepest internal node (>= 2 children) gives LRS.

```

```

129 string longestRepeated(SuffixTree& st) {
130     string best = "";
131     function<void(int, int)> dfs = [&](int v, int depth) {
132         int childCount = 0;
133         for (int c = 0; c < 26; c++) {
134             if (st.t[v].ch[c] == -1) continue;
135             childCount++;
136             int nxt = st.t[v].ch[c];
137             dfs(nxt, depth + st.t[nxt].len(st.n - 1));
138         }
139         // internal node with depth > best
140         if (childCount > 0 && depth > (int)best.size()) {
141             // walk back up to find the string — simplified via start pointer
142             int start = st.t[v].start - (depth - st.t[v].len(st.n - 1));
143             best = st.s.substr(max(0, start), depth);
144         }
145     };
146     dfs(0, 0);
147     return best; // removes trailing '$' if present
148 }
149
150 // --- Count distinct substrings ---
151 // Each edge contributes (edgeLen) new distinct substrings.
152 // Total = sum of all edge lengths in the tree.
153 long long countDistinct(SuffixTree& st) {
154     long long total = 0;
155     // node 0 is root (no parent edge), skip
156     for (int v = 1; v < (int)st.t.size(); v++)
157         total += st.t[v].len(st.n - 1);
158     return total - 1; // subtract 1 for the empty string
159 }
160
161 // --- Longest Common Substring of two strings ---
162 // Build suffix tree of S1 + '#' + S2 + '$'
163 // Find deepest internal node whose subtree has leaves from both S1 and S2.
164 string longestCommon(const string& s1, const string& s2) {
165     SuffixTree st;
166     string combined = s1 + "#" + s2 + "$";
167     st.build(combined);
168     int n1 = s1.size();
169     string best = "";
170
171     // DFS: returns {has_s1_leaf, has_s2_leaf}
172     function<pair<bool, bool>(int, int)> dfs = [&](int v, int dep) -> pair<bool, bool> {
173         bool has1 = false, has2 = false;
174         bool leaf = true;
175
176         for (int c = 0; c < 26; c++) {
177             if (st.t[v].ch[c] == -1) continue;
178             leaf = false;
179             int nxt = st.t[v].ch[c];
180             auto [h1, h2] = dfs(nxt, dep + st.t[nxt].len(st.n - 1));
181             has1 |= h1;
182             has2 |= h2;
183         }
184
185         if (leaf) {
186             int sufIdx = st.n - 1 - dep;
187             has1 = (sufIdx < (int)n1);
188             has2 = (sufIdx > (int)n1); // > n1 skips the '#'
189         }
190
191         // internal node with leaves from both strings
192         if (!leaf && has1 && has2 && dep > (int)best.size()) {
193             // recover substring: edge ends at t[v].end, go back 'dep' chars
194             int endPos = st.t[v].end;
195             int startPos = endPos - dep;
196             if (startPos >= 0 && endPos <= (int)st.s.size())
197                 best = st.s.substr(startPos, dep);
198         }
199         return {has1, has2};
200     };
201
202     dfs(0, 0);
203     return best;
204 }
205
206 // --- Pattern matching demo ---
207
208 // Ejemplo de uso anterior (main comentado para compilar con la plantilla):
209 // int main() {
210 //     SuffixTree st;
211 //     st.build("banana");

```

```

212 // cout << longestRepeated(st) << "\n";
213 // cout << countDistinct(st) << "\n";
214 // cout << longestCommon("banana", "ananas") << "\n";
215 // for (int pos : st.search("ana")) cout << pos << " ";
216 // cout << "\n";
217 // }

```

Trie

```

1 struct Trie {
2
3     static const int A = 26; // alphabet size
4     struct Node {
5         int nxt[A], cnt = 0, end = 0; // cnt: words passing here, end: words ending here
6         Node() { memset(nxt, -1, sizeof nxt); }
7     };
8     vector<Node> t;
9     Trie() : t(1) {}
10    void insert(const string& s) {
11        int x = 0;
12        for (char c : s) {
13            int k = c - 'a';
14            if (t[x].nxt[k] < 0) { t[x].nxt[k] = t.size(); t.emplace_back(); }
15            x = t[x].nxt[k];
16            t[x].cnt++;
17        }
18        t[x].end++;
19    }
20    int find(const string& s) { // node of s, or -1
21        int x = 0;
22        for (char c : s) {
23            x = t[x].nxt[c - 'a'];
24            if (x < 0) return -1;
25        }
26        return x;
27    }
28    int count(const string& s) { int x = find(s); return x < 0 ? 0 : t[x].end; } // exact matches
29    int prefix(const string& s) { int x = find(s); return x < 0 ? 0 : t[x].cnt; } // words with
    prefix s
30    bool erase(const string& s) { // removes one copy, false if absent
31        if (!count(s)) return false;
32        int x = 0;
33        for (char c : s) { x = t[x].nxt[c - 'a']; t[x].cnt--; }
34        t[x].end--;
35        return true;
36    }
37 };

```

Z Function

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 vector<int> z_function(const string& s) {
5     int n = s.size();
6     vector<int> z(n, 0);
7     z[0] = n;
8     int l = 0, r = 0;
9     for (int i = 1; i < n; i++) {
10         if (i < r) z[i] = min(z[i - l], r - i);
11         while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
12         if (i + z[i] > r) { l = i; r = i + z[i]; }
13     }
14     return z;
15 }
16
17 // -- 1. Pattern matching: find all occurrences of P in T -----
18 // Concatenate P + '#' + T. Any position i in the T-part where z[i] == |P|
19 // corresponds to a match at T[i - |P| - 1].
20 vector<int> find_all(const string& P, const string& T) {
21     string s = P + '#' + T;
22     auto z = z_function(s);
23     int m = P.size();
24     vector<int> matches;
25     for (int i = m + 1; i < (int)s.size(); i++)
26         if (z[i] == m) matches.push_back(i - m - 1);
27     return matches;
28 }
29
30 // -- 2. Shortest period of a string -----
31 // The shortest period p satisfies: s[0..n-p-1] == s[p..n-1],
32 // i.e. z[p] + p == n (the suffix at p matches the prefix and reaches the end).

```

```

33 int shortest_period(const string& s) {
34     int n = s.size();
35     auto z = z_function(s);
36     for (int p = 1; p < n; p++)
37         if (z[p] + p == n) return p;
38     return n; // no period shorter than n
39 }
40
41 // -- 3. Longest border (prefix = suffix, proper) -----
42 // A border of length k exists iff z[n-k] == k.
43 int longest_border(const string& s) {
44     int n = s.size();
45     auto z = z_function(s);
46     for (int k = n - 1; k >= 1; k--)
47         if (z[n - k] == k) return k;
48     return 0;
49 }
50
51 // Ejemplo de uso anterior (main comentado para compilar con la plantilla):
52 // int main() {
53 //     string text = "abacaba", pattern = "aba";
54 //     for (int pos : find_all(pattern, text)) cout << pos << " ";
55 //     cout << "\nperiod = " << shortest_period(text) << "\n";
56 // }

```